Bases de données et analyses fonctionnelles

Bioinformatique I (BIF-7900)

Antoine Bodein

12 novembre 2026



1

INTRODUCTION

Objectifs

Fin du TP3 de statistiques : `airway`, dexaméthasone contre non traité.

```
res
#> # A tibble: 22,369 × 6
#>   gene            baseMean log2FoldChange   stat     pvalue      padj
#>   <chr>              <dbl>          <dbl>  <dbl>      <dbl>     <dbl>
#> 1 ENSG00000000003    709.         -0.381  -3.79  0.000153   0.00129
#> 2 ENSG00000000419    520.          0.207   1.84  0.0650     0.195
#> 3 ENSG00000000457    237.          0.0380  0.266 0.790      0.910
#> 4 ENSG00000000460     57.9        -0.0885 -0.311 0.756      0.893
#> 5 ENSG00000000971   5817.          0.426   4.80  0.00000157 0.0000206
#> 6 ENSG00000001036   1282.         -0.241  -2.71  0.00676    0.0335
#> # ℹ 22,363 more rows
```

1018 gènes avec `padj` < 0.05 et |log2FC| > 1. Des identifiants ENSG.... **Et maintenant ?**

1. Savoir **où chercher** une information biologique, et **ce que vaut** cette information
2. Comprendre une **ontologie** (Gene Ontology) et ses trois branches
3. **Convertir** des identifiants de gènes sans en perdre ni en dupliquer en silence
4. Mener et lire une analyse de **sur-représentation** (ORA)
5. Mener et lire une **GSEA**, et savoir quand préférer l'une ou l'autre

Questions

- Qu'est-ce que le gène *BRCA1* ?
- Dans quelle maladie est-il impliqué ?
- Pour quelle protéine code-t-il ?
- Quel est son identifiant ? **Lequel ?**

> Chacune de ces questions a sa **base de données**, et chaque base son **identifiant**. Aujourd'hui : les trouver, les relier, puis passer d'une liste de 1000 gènes à quelques **fonctions** biologiques.

Plan

1. Introduction
2. Les bases de données en biologie
3. Les ontologies : un vocabulaire commun
4. Identifiants et conversion → **TP1**
5. L'analyse de sur-représentation (ORA)
6. La GSEA → **TP2**

Les données du jour

Les résultats DESeq2 du TP3 de statistiques, un gène par ligne :

```
res <- read_tsv("/home/public/enrichissement/airway_deseq2.tsv")
```

Colonne	Contenu
`gene`	identifiant **Ensembl** du gène (`ENSG...`)
`baseMean`	expression moyenne normalisée
`log2FoldChange`	traité **par rapport au** non traité (`dex trt vs untrt`)
`stat`	statistique de Wald : log2FC / erreur standard, **signée**
`pvalue`, `padj`	p-value brute, p-value ajustée (BH) ; `NA` = gène non testé

Cellules musculaires lisses des voies respiratoires, 4 donneurs, traitées ou non à la dexaméthasone (Himes *et al.*, 2014). Plan `~ cell + dex`, `untrt` en référence.

2

LES BASES DE DONNÉES EN BIOLOGIE

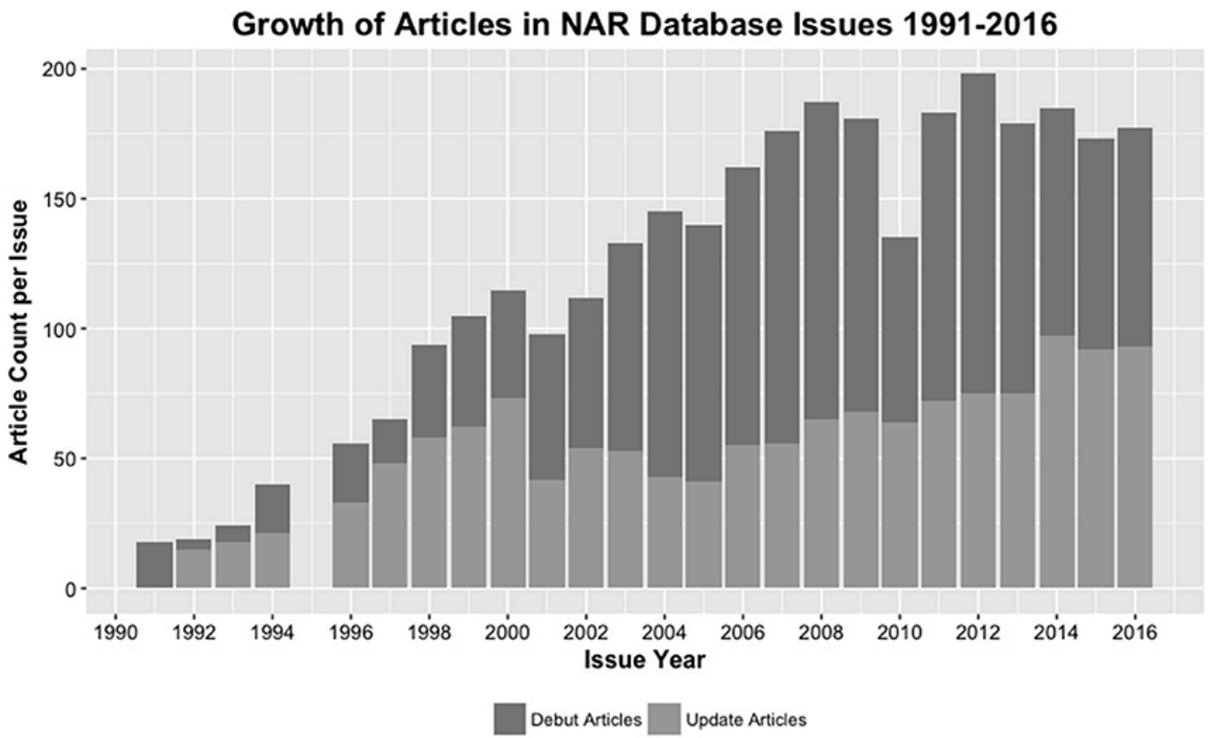
Qu’est-ce qu’une banque de données ?

- Une **archive** qui stocke et organise l’information pour qu’on la retrouve selon divers critères
- L’information vient d’**expériences** (séquençage, structures, puces), de **publications** et d’**analyses informatiques** (prédictions)
- La plupart sont **interconnectées** : un vaste réseau de liens, source de richesse… et de confusion



> Données → modèles → connaissances. Personne ne lit 20 000 gènes : les bases de données permettent de **réutiliser** ce que d’autres ont mesuré et décrit.

Des milliers de bases de données



Chaque année, la revue *Nucleic Acids Research* consacre un numéro aux bases de données :

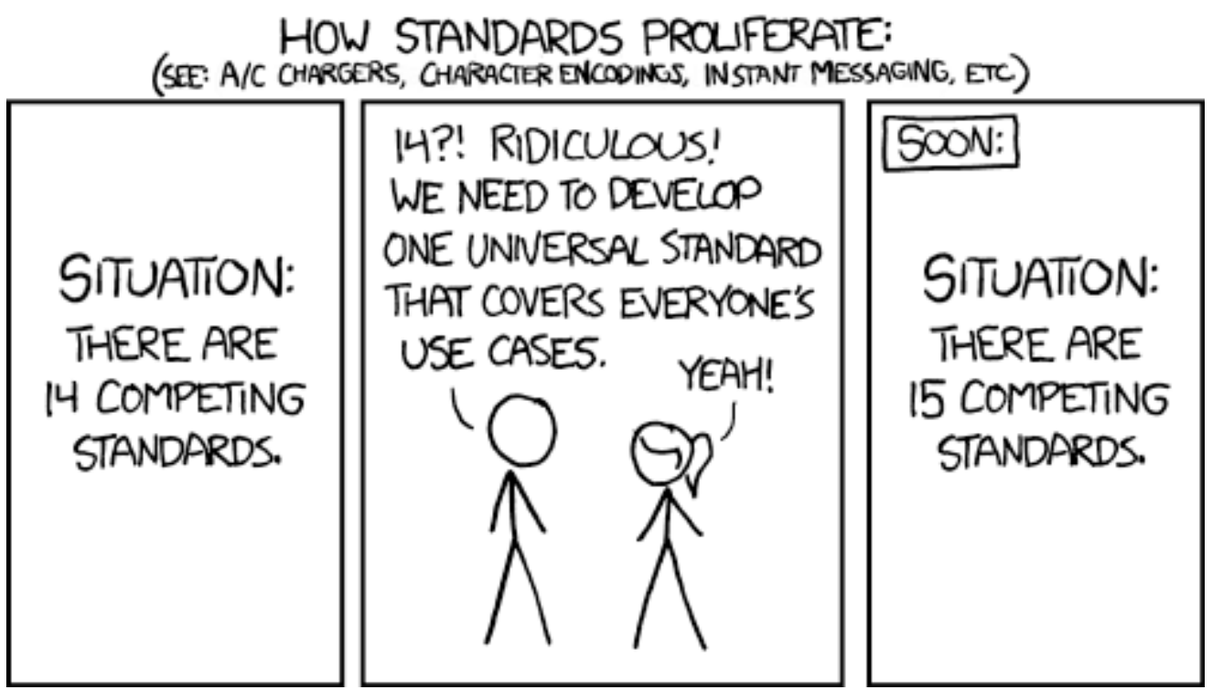
- 2000 : 230 bases
- 2006 : 856
- 2021 : 1 697
- 2023 : 1 764

Chaque année : des **nouvelles**, des **mises à jour**… et des bases **retirées** ou plus maintenues.

Rigden et Fernández, *NAR Database Issue*, éditoriaux annuels.

D'où viennent ces données ?

- **Séquences** : génomes, transcrits, protéines
- **Expression** : puces, RNA-seq, *single-cell*
- **Structures** : protéines, complexes
- **Interactions et voies**
- **Variants** et **maladies**
- **Ontologies, littérature**



xkcd 927, « Standards » (R. Munroe) : chaque nouvelle base veut unifier les autres… et en ajoute une.

Classer les bases de données

Critère	Exemples
Étendue	généraliste (GenBank, UniProt) ou spécialisée (WormBase, une maladie, un organisme)
Niveau	**primaire** (données brutes, déposées) ou **secondaire** (données traitées, interprétées)
Curation	annotation **automatique** ou **experte** (manuelle)
Type de données	ADN, ARN, protéine, expression, voies, maladies, nomenclature, littérature, ontologies

Zou *et al.* (2015), *Biological databases for human research*, *Genomics Proteomics Bioinformatics*.

> Avant d'utiliser une base : **qui** l'alimente, **comment** est-elle vérifiée, **quand** a-t-elle été mise à jour ?

Les bases primaires

Elles contiennent les résultats expérimentaux **bruts**, tels que **déposés** par les chercheurs.

- **INSDC** : GenBank (NCBI, États-Unis), ENA (EMBL-EBI, Europe), DDBJ (Japon). Un consortium : elles **échangent leurs données chaque jour** et contiennent les mêmes séquences
- **SRA** (*Sequence Read Archive*) : les lectures NGS brutes. Le dépôt exige de décrire les échantillons et le plan d'expérience
- **GEO** (NCBI) : données d'expression (puces, RNA-seq), avec leurs métadonnées
- **PDB** : les structures 3D de macromolécules, avec les coordonnées des atomes

> Le **dépôt** des séquences et des données brutes est une condition de publication dans la plupart des revues. C'est ce qui permet de **refaire** une analyse : `airway` est déposé dans GEO (GSE52778).

Les bases secondaires

Elles **transforment** l'information brute en connaissances plus élaborées, par des programmes ou par des experts.

UniProtKB : les protéines

- **Swiss-Prot** : revue **manuellement** par des experts
- **TrEMBL** : annotée **automatiquement** (traduction des séquences de l'INSDC)
- TrEMBL est des centaines de fois plus grand que Swiss-Prot

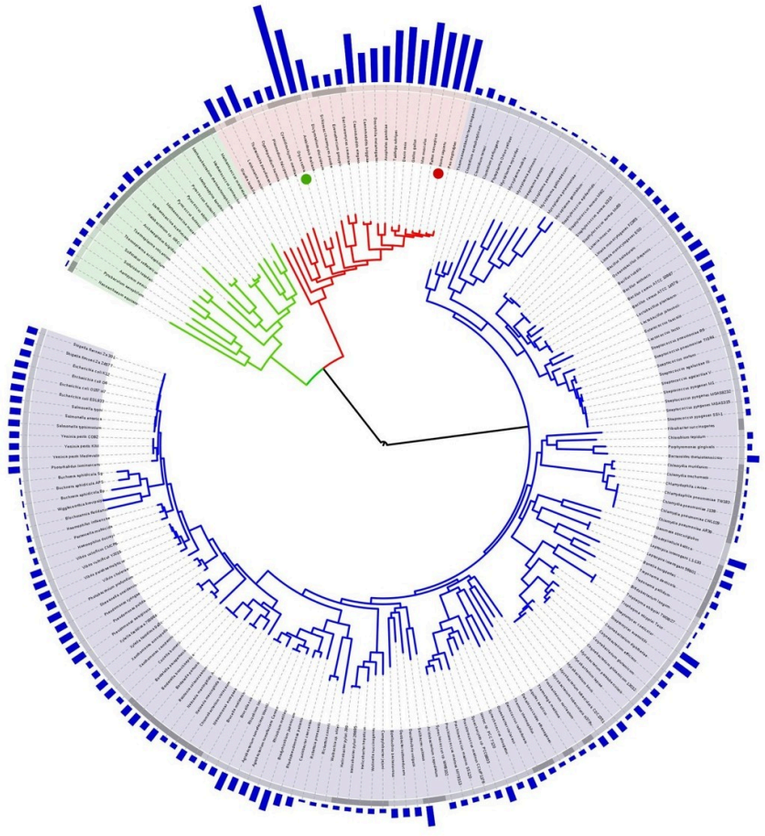
Voies et réseaux

- **Reactome** : réactions et voies, curation manuelle, libre
- **KEGG** : voies métaboliques et de signalisation ; licence payante pour le téléchargement complet
- **Gene Ontology** : les fonctions des gènes (section 3)

> Une base secondaire contient souvent des informations **prédites** par un programme. Une analyse fondée sur des données incorrectes conduit à des conclusions erronées, **sans aucun message d'erreur**.

Les limites des bases de données

- Des **erreurs** d'annotation et de séquençage, qui se **propagent** d'une base à l'autre par annotation automatique
- De la **redondance** : la même séquence sous plusieurs entrées
- Des **biais** : certaines branches de l'arbre du vivant (l'humain, la souris, les pathogènes) sont sur-représentées ; les gènes déjà étudiés sont les mieux annotés
- Des **versions** : la base change à chaque mise à jour, et vos résultats avec



Remède partiel : un **vocabulaire contrôlé** (Gene Ontology, MeSH) au lieu de texte libre, et noter **la version** de chaque base utilisée.

Interroger une base en ligne

- Un **portail** web avec des opérateurs booléens : `AND`, `OR`, `NOT`, des parenthèses, des guillemets pour une expression exacte
- Recherche **avancée** : chercher dans un **champ** précis (nom du gène, organisme, statut revu)
- Exemple sur https://www.uniprot.org :

```
BRCA1                                          → des milliers de résultats, toutes espèces
gene:BRCA1 AND organism_id:9606                → l'humain seulement
gene:BRCA1 AND organism_id:9606 AND reviewed:true → l'entrée Swiss-Prot P38398
```

> Chaque filtre ajouté correspond à un champ d'une base **relationnelle**. C'est la même logique que `filter()` sur un tableau.

✋ À vous : primaire ou secondaire ?

Pour chaque ressource, primaire ou secondaire ? Curation manuelle ou automatique ?

1. Les lectures FASTQ d'`airway` dans SRA
2. L'entrée Swiss-Prot P38398 (BRCA1)
3. Une entrée TrEMBL d'une protéine de bactérie séquencée en 2025
4. Le terme GO « *response to glucocorticoid* » associé à un gène avec le code d'évidence `IEA`

1. **Primaire** : les données brutes déposées
2. **Secondaire**, curation **manuelle** (experts) : la référence
3. **Secondaire**, **automatique** : une traduction de séquence, des fonctions prédites
4. **Secondaire**, **automatique** : `IEA` = *Inferred from Electronic Annotation*, personne ne l'a vérifiée (section suivante)

3

LES ONTOLOGIES : UN VOCABULAIRE COMMUN

Pourquoi une ontologie ?

- Un même processus décrit avec des mots différents : « apoptose », « mort cellulaire programmée », « *programmed cell death* »
- Une **ontologie** : un ensemble de **concepts** (termes) définis, chacun avec un **identifiant** stable, reliés par des **relations**
  - **hiérarchiques** : « est un » (*is a*), du général au particulier
  - **sémantiques** : « fait partie de » (*part of*), « régule » (*regulates*)
- Tout le monde parle **la même langue**, et une machine peut raisonner sur les termes

> `GO:0051384` veut dire « *response to glucocorticoid* » pour tous les logiciels, dans toutes les langues, quel que soit le nom qu'un auteur lui donne.

Gene Ontology (GO)

Trois branches indépendantes :

- **MF**, *Molecular Function* : l'activité moléculaire du produit du gène (« activité kinase »)
- **CC**, *Cellular Component* : où il agit (« mitochondrie »)
- **BP**, *Biological Process* : le processus auquel il contribue (« réponse aux glucocorticoïdes »)

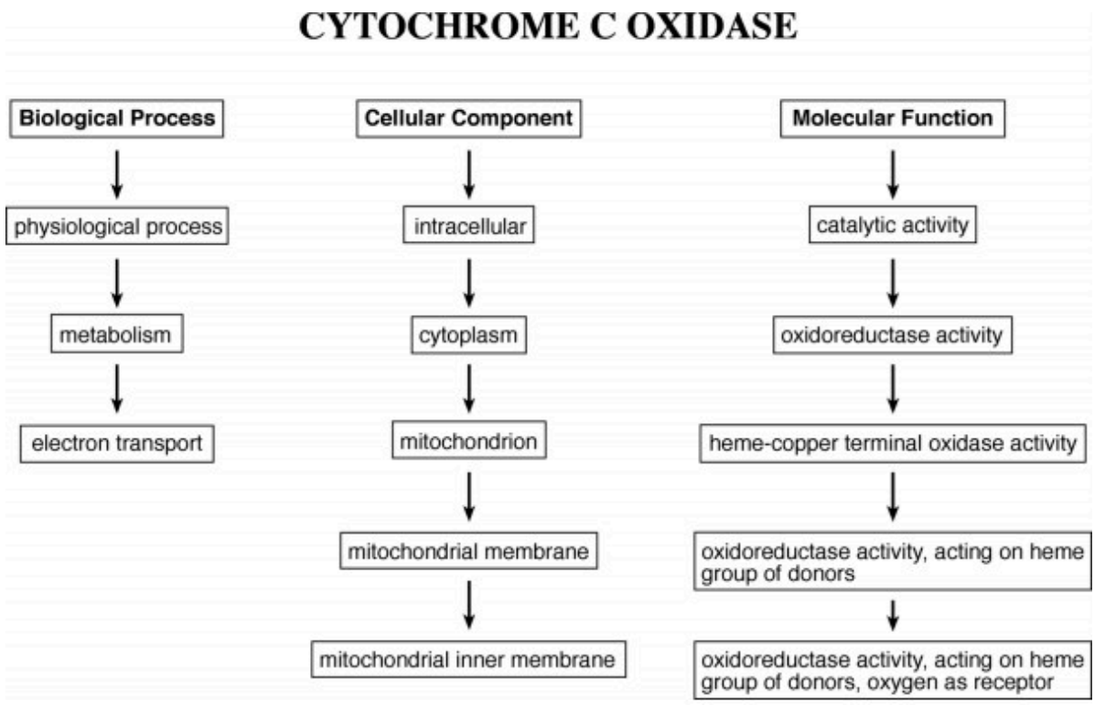


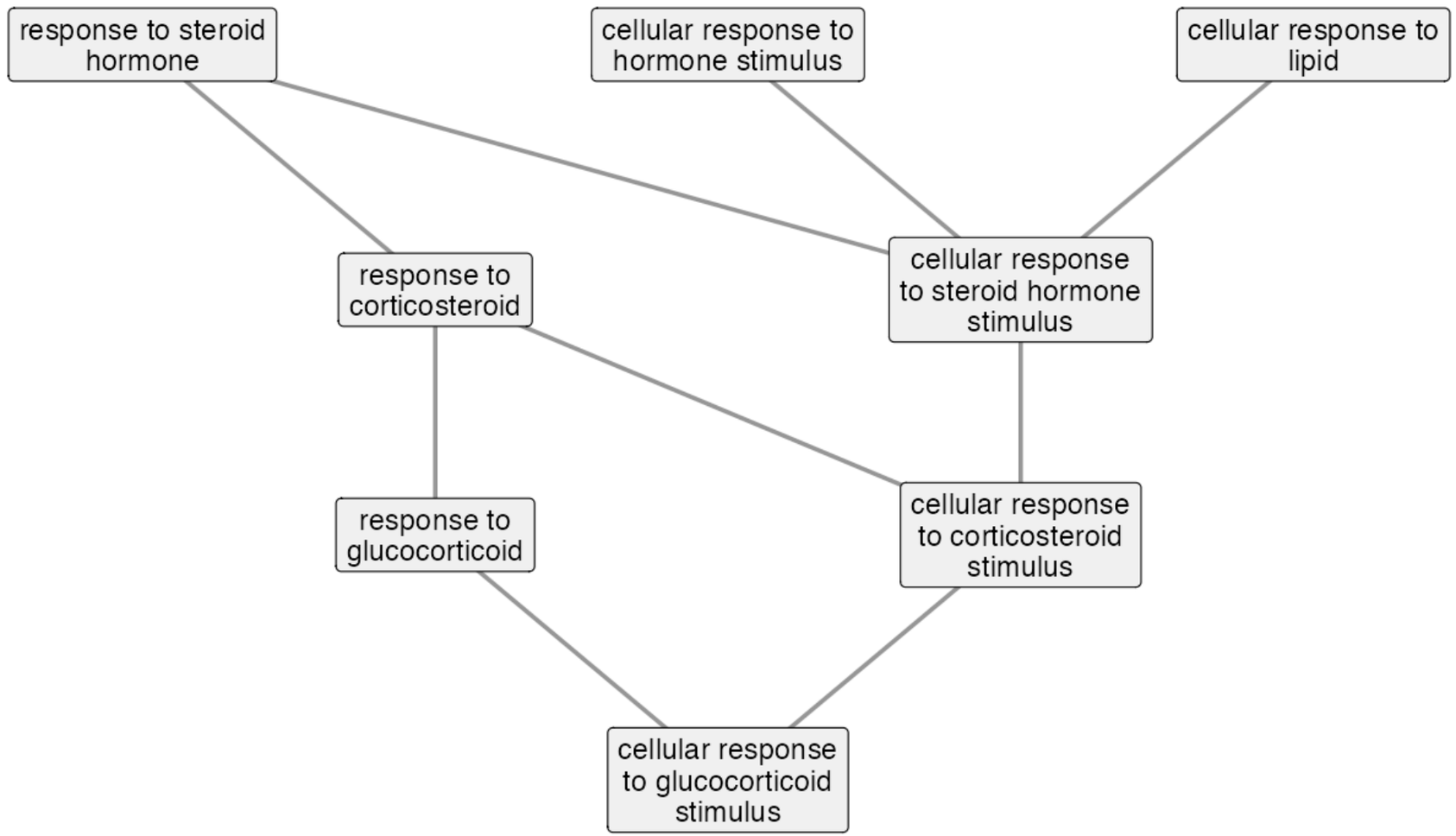
Figure 17.5: Example of GO annotation for cytochrome *c* oxidase. The functional and structural terms are arranged in three categories with a number of hierarchies indicating the levels of conceptual associations of protein functions.

Exemple : la cytochrome c oxydase dans les trois branches.

Dans la version de `GO.db` installée : 26 091 termes BP, 10 154 MF, 4 022 CC.

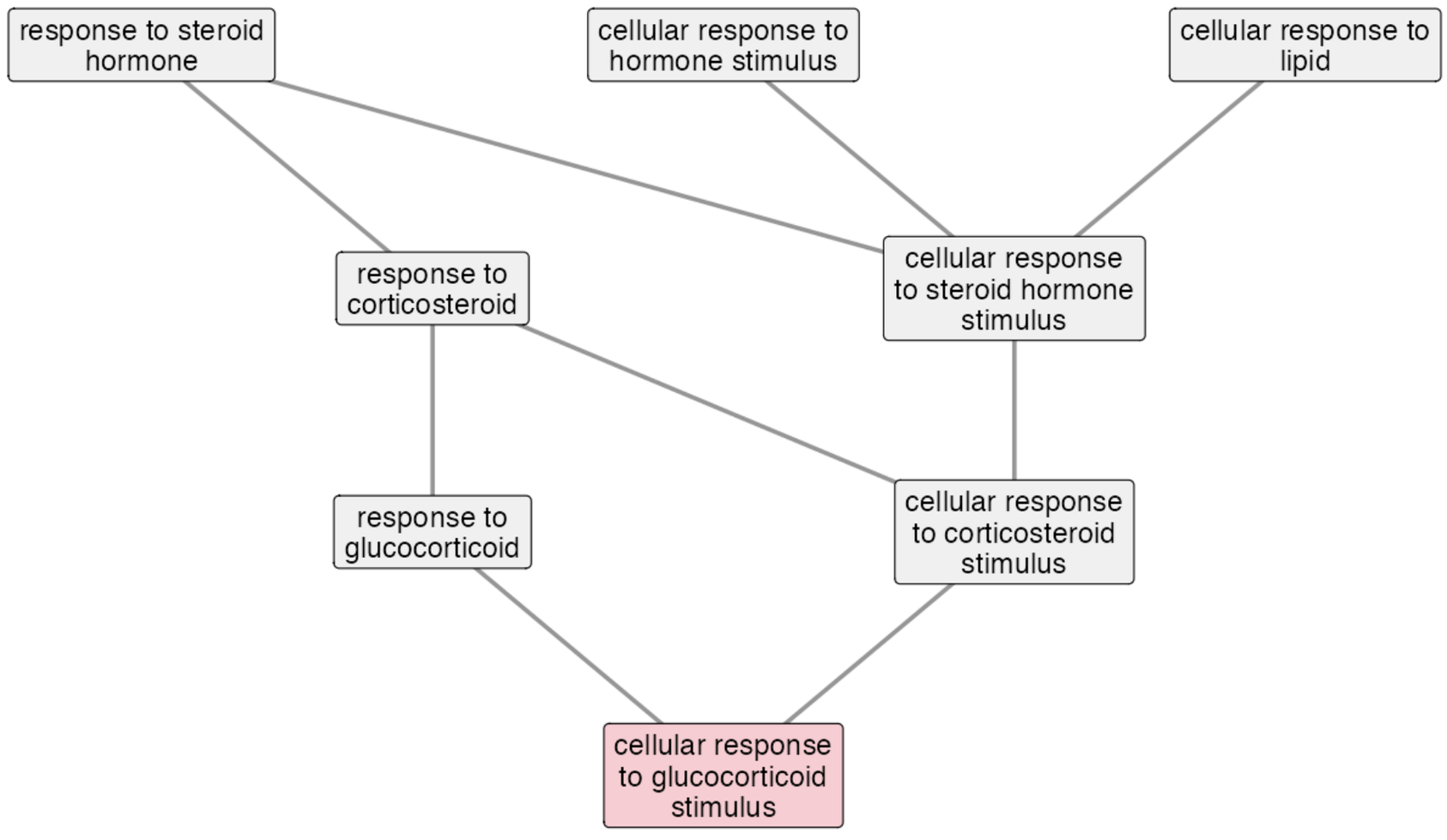
Un graphe, pas un arbre

Un terme précis (en bas) et ses ancêtres : du **particulier** au **général**. Un terme peut avoir **plusieurs parents**.



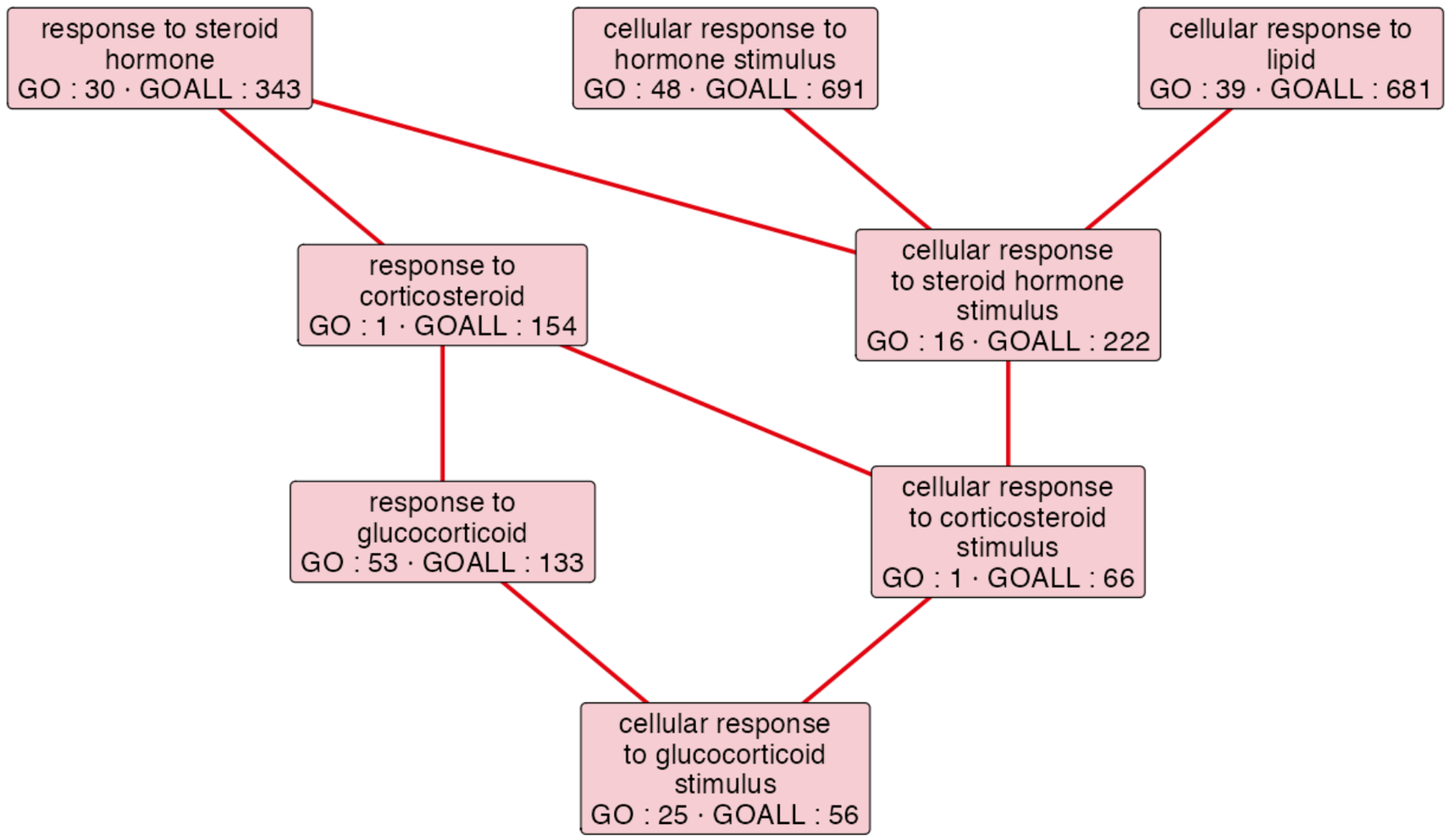
Un graphe, pas un arbre

Un gène est annoté **seulement** à *cellular response to glucocorticoid stimulus*.

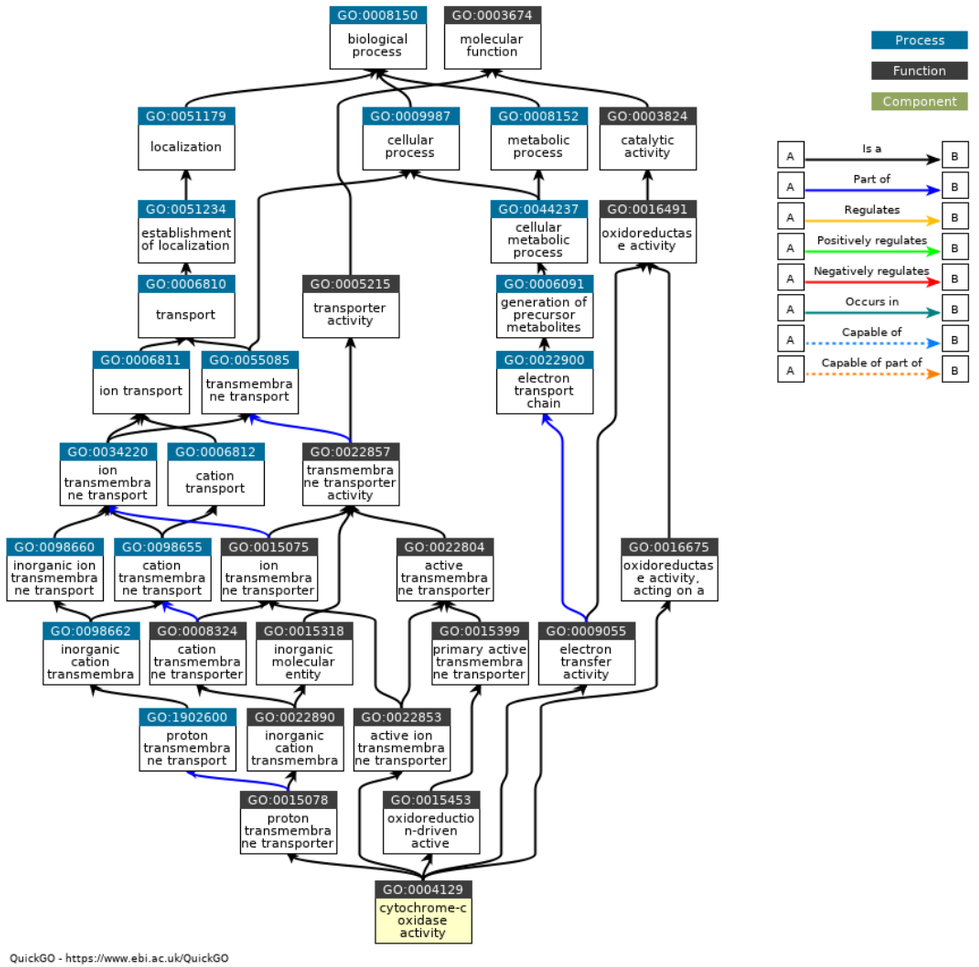


Un graphe, pas un arbre

Règle du chemin vrai : il compte aussi pour **tous les ancêtres**. GO = gènes annotés directement ; GOALL = avec les descendants.



À l'échelle de GO



- Des dizaines de milliers de termes, reliés en un **graphe orienté acyclique** (DAG)
- Relations conservées dans la version de base (`go-basic`) : *is a*, *part of*, *regulates* ; toutes respectent la règle du chemin vrai
- Dans `org.Hs.eg.db` : la clé `GO` donne les annotations **directes**, la clé `GOALL` les annotations **propagées**. Pour l'enrichissement, c'est `GOALL` qui compte

> Conséquence pour l'enrichissement : un parent et ses enfants partagent **les mêmes gènes**. Les listes de résultats sont très **redondantes** (`simplify()` de `clusterProfiler` en retire une partie).

Les codes d'évidence

Chaque annotation gène → terme GO porte un **code d'évidence** : sur quoi repose-t-elle ?

Code	Signification	Fiabilité
`EXP`, `IDA`, `IMP`, `IPI`	expérience publiée (directe, mutant, interaction)	forte
`IBA`, `ISS`	inférée par phylogénie ou similarité de séquence	moyenne
`TAS`	*Traceable Author Statement* : affirmé dans un article	variable
`IEA`	*Inferred from Electronic Annotation* : automatique, **non revue**	à vérifier

SPARCL1 : 16 annotations GO, dont 7 `IEA`. Les gènes les plus étudiés ont le plus d'annotations expérimentales : **l'ontologie reflète aussi ce qu'on a déjà cherché**.

MeSH et les autres ontologies

- **MeSH** (*Medical Subject Headings*) : vocabulaire contrôlé et hiérarchisé de la NLM, qui indexe les articles de **PubMed**
- **Disease Ontology** (DO), **Human Phenotype Ontology** (HPO) : maladies, phénotypes
- **Sequence Ontology**, **ChEBI** (molécules), **Cell Ontology**, **Uberon** (anatomie)…
- Le portail **OBO Foundry** fédère des ontologies biomédicales construites selon les mêmes principes

> Toutes s'utilisent de la même façon pour l'enrichissement : un **ensemble de gènes** par terme. Seules la source et la qualité des annotations changent.

Interroger GO dans R

Tous les packages d'annotation `.db` de Bioconductor s'interrogent **de la même manière** : `keytypes()`, `columns()`, `select()`.

```
terme <- AnnotationDbi::select(GO.db, keys = "GO:0051384", keytype = "GOID",
                               columns = c("TERM", "ONTOLOGY", "DEFINITION"))
terme$TERM
#> [1] "response to glucocorticoid"
terme$ONTOLOGY
#> [1] "BP"
strwrap(terme$DEFINITION, 70)
#> [1] "Any process that results in a change in state or activity of a cell"
#> [2] "or an organism (in terms of movement, secretion, enzyme production,"
#> [3] "gene expression, etc.) as a result of a glucocorticoid stimulus."
#> [4] "Glucocorticoids are hormonal C21 corticosteroids synthesized from"
#> [5] "cholesterol with the ability to bind with the cortisol receptor and"
#> [6] "trigger similar effects. Glucocorticoids act primarily on"
#> [7] "carbohydrate and protein metabolism, and have anti-inflammatory"
#> [8] "effects."
```

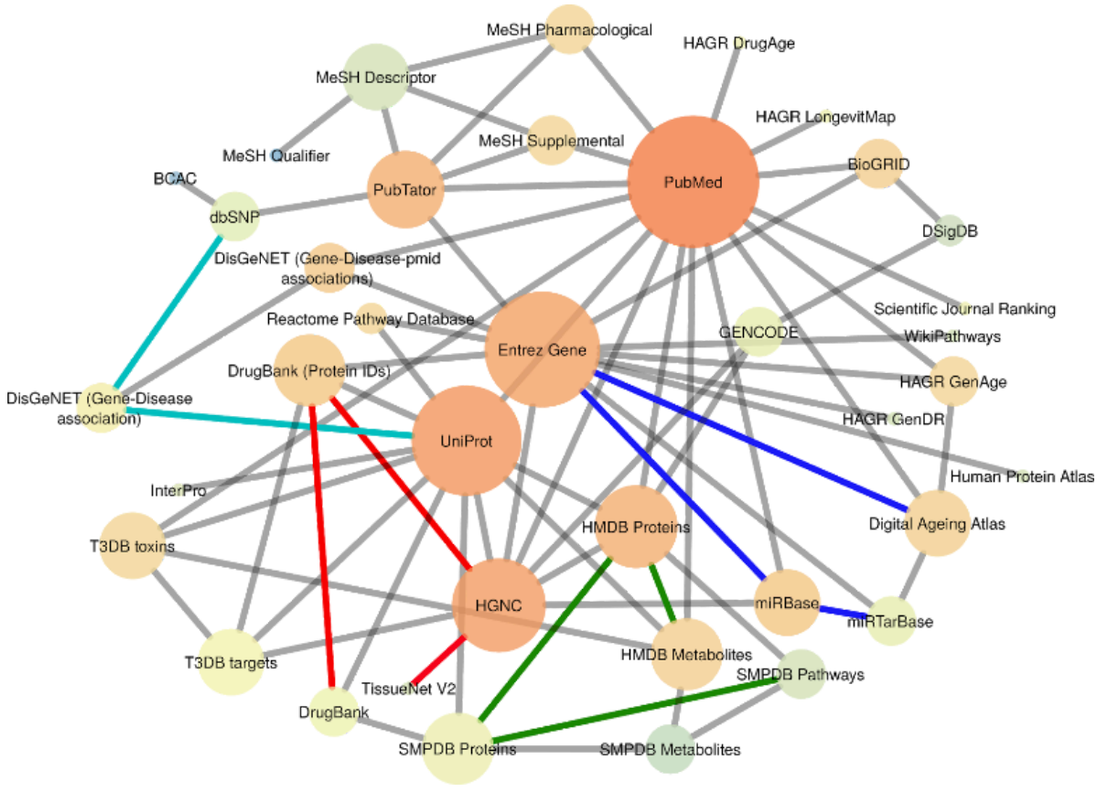
```
as.list(GOBPPARENTS["GO:0051384"])   # parents dans la branche BP
#> $`GO:0051384`
#>         isa
#> "GO:0031960"
```

4

IDENTIFIANTS ET CONVERSION

Un gène, plusieurs identifiants

Système	*BRCA1*	Géré par
Symbole	BRCA1	HGNC
Entrez Gene	672	NCBI
Ensembl	ENSG00000012048	EMBL-EBI
UniProt	P38398	UniProt



Ongaro-Carcy *et al.* (2021), *Bioinformatics* (KibioR).

Chaque base a **son** identifiant. Les liens entre identifiants sont **eux-mêmes** une base de données, avec sa version, ses erreurs et ses trous.

Le symbole officiel (HGNC)

- Attribué par le **HUGO Gene Nomenclature Committee** pour l'humain (MGI pour la souris : `Brca1`)
- Lisible, mais **pas stable** : il change quand la fonction est mieux comprise ou pour éviter une ambiguïté
- En 2020, *SEPT2* est devenu *SEPTIN2* et *MARCH1* est devenu *MARCHF1* : Excel les convertissait en **dates** (2-sept., 1-mars)

```
AnnotationDbi::select(org.Hs.eg.db, keys = c("SEPT2", "MARCH1"),
                      keytype = "ALIAS", columns = "SYMBOL")
#>     ALIAS  SYMBOL
#> 1   SEPT2 SEPTIN2
#> 2   SEPT2 SEPTIN6
#> 3 MARCH1 MARCHF1
```

Environ **un article sur cinq** fournissant des listes de gènes dans Excel contenait des noms convertis en dates (Ziemann *et al.*, 2016, *Genome Biology*). Un ancien symbole peut aussi désigner **deux** gènes actuels : `SEPT2` → `SEPTIN2` **et** `SEPTIN6`.

Entrez Gene et Ensembl

Entrez Gene ID (NCBI)

- un **entier** : `672`
- unique, stable, propre à une espèce
- utilisé par de nombreux outils d'enrichissement (KEGG, `clusterProfiler`)

Ensembl ID (EMBL-EBI)

`ENS` + espèce + type + numéro + version

- `ENSG00000012048` : gène humain
- `ENSMUSG...` : gène de souris
- `ENST...` : transcrit ; `ENSP...` : protéine
- `ENSG00000012048.23` : la **version**

```
ids <- c("ENSG00000152583.13", "ENSG00000096060.15")
sub("\\..*", "", ids)    # retirer la version avant de convertir
#> [1] "ENSG00000152583" "ENSG00000096060"
```

Si **tous** les identifiants portent une version : une erreur (`None of the keys entered are valid keys`). Si **quelques-uns** seulement : des `NA`, **sans erreur**.

Où convertir ?

- En ligne : **UniProt ID mapping** (https://www.uniprot.org/id-mapping), **g:Profiler g:Convert**, **BioMart** (Ensembl)
- Dans R, avec **BioMart** : interroge le serveur d'Ensembl, toujours à jour, **besoin d'Internet**

```
library(biomaRt)
ensembl <- useEnsembl(biomart = "genes", dataset = "hsapiens_gene_ensembl")
getBM(attributes = c("ensembl_gene_id", "hgnc_symbol", "entrezgene_id"),   # ce que l'on veut
      filters    = "entrezgene_id",                                         # ce que l'on a
      values     = c(672, 5468, 7157),                                      # nos valeurs
      mart       = ensembl)
```

- Dans R, **hors ligne** : les packages d'annotation de **Bioconductor** (`org.Hs.eg.db`), une version **figée**

Bioconductor

- Un dépôt de packages R pour la bioinformatique, comme le CRAN, avec une **révision** plus stricte
- **Deux versions par an** (avril et octobre), chacune liée à une version de R
- Trois grandes familles :
  - **Software** : l'analyse (`DESeq2`, `clusterProfiler`, `limma`)
  - **AnnotationData** : génomes, annotations (`org.Hs.eg.db`, `GO.db`)
  - **ExperimentData** : des jeux de données prêts à l'emploi (`airway`)

```
install.packages("BiocManager")
BiocManager::install(c("org.Hs.eg.db", "clusterProfiler"))
```

`org.Hs.eg.db` : une base dans un package

Une base **SQLite** : les correspondances entre identifiants, et les annotations GO, pour l'humain (Hs), centrée sur Entrez (eg).

```
keytypes(org.Hs.eg.db)
#>  [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
#>  [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
#> [11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
#> [16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
#> [21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
#> [26] "UNIPROT"
```

Sources de la version installée : Entrez 2025-Sep24, GO 2025-07-22, Ensembl 2025-Sep03… et KEGG **2011-Mar15** : la licence de KEGG a changé en 2011, cette colonne n'a plus jamais été mise à jour.

`select()` : convertir

```
AnnotationDbi::select(org.Hs.eg.db, keys = mes_ids, keytype = “ENSEMBL”,
columns = c(“SYMBOL”, “ENTREZID”))
```

la base | les valeurs que l’on a | leur type | ce que l’on veut

mes_ids : 4 gènes

ENSEMBL
ENSG00000152583
ENSG00000004866
ENSG00000152268
ENSG00000096060

`select()` : convertir

```
AnnotationDbi::select(org.Hs.eg.db, keys = mes_ids, keytype = “ENSEMBL”,
columns = c(“SYMBOL”, “ENTREZID”))
```

la base | les valeurs que l’on a | leur type | ce que l’on veut

résultat : 5 lignes

ENSEMBL	SYMBOL	ENTREZID
ENSG00000152583	SPARCL1	8404
ENSG00000004866	ST7	7982
ENSG00000004866	ST7-OT3	93655
~~ENSG00000152268~~	~~NA~~	~~NA~~
ENSG00000096060	FKBP5	2289

- **Un-à-plusieurs** : `ENSG00000004866` donne **deux** lignes (*ST7* et *ST7-OT3*). Message : `'select()' returned 1:many mapping`
- **Aucune correspondance** : `NA`, sans erreur. Le tableau n’a plus le même nombre de lignes que `mes_ids`

`mapIds()` : une valeur par identifiant

```
mapIds(org.Hs.eg.db, keys = mes_ids, keytype = "ENSEMBL", column = "SYMBOL")
#> ENSG00000152583 ENSG00000004866 ENSG00000152268 ENSG00000096060
#>       "SPARCL1"            "ST7"              NA         "FKBP5"
```

- **Un seul** résultat par clé, dans **le même ordre** : on peut l'ajouter comme colonne avec `mutate()`
- `multiVals = "first"` (défaut) : en cas de choix, garde le **premier**… et ne dit pas lequel il a écarté
- `multiVals = "list"` : tous les résultats, dans une liste

```
mapIds(org.Hs.eg.db, keys = "ENSG00000004866", keytype = "ENSEMBL",
       column = "SYMBOL", multiVals = "list")
#> $ENSG00000004866
#> [1] "ST7"     "ST7-OT3"
```

Sur nos 1018 gènes DE

```r
de_annot <- de |>
  mutate(symbole = mapIds(org.Hs.eg.db, keys = gene, keytype = "ENSEMBL", column = "SYMBOL"))
sum(is.na(de_annot$symbole))
#> [1] 82
de_annot |> arrange(desc(log2FoldChange)) |> filter(!is.na(symbole)) |> head(5)
#> # A tibble: 5 × 7
#>   gene            baseMean log2FoldChange  stat   pvalue     padj symbole
#>   <chr>              <dbl>          <dbl> <dbl>    <dbl>    <dbl> <chr>
#> 1 ENSG00000179593    67.2            9.51  9.02 1.85e-19 1.18e-17 ALOX15B
#> 2 ENSG00000109906   385.             7.35 13.7  9.14e-43 2.32e-40 ZBTB16
#> 3 ENSG00000250978    56.3            6.33  9.36 8.33e-21 6.03e-19 LOC101928819
#> 4 ENSG00000132518     5.65           5.88  4.47 7.72e- 6 8.86e- 5 GUCY2D
#> 5 ENSG00000127954   286.             5.21 10.6  4.55e-26 5.06e-24 STEAP4
```

82 gènes DE sans symbole : des gènes **réels**, mais absents de cette version de la base (ou non codants). Ils disparaîtront **en silence** de toute analyse qui passe par les symboles.

Piège : `select()` masqué

`library(org.Hs.eg.db)` charge `AnnotationDbi`, qui a **sa propre** fonction `select()` :

```
res |> select(gene, padj)
#> # A tibble: 22,369 × 2
#>   gene                padj
#>   <chr>              <dbl>
#> 1 ENSG00000000003 0.00129
#> 2 ENSG00000000419 0.195
#> 3 ENSG00000000457 0.910
#> 4 ENSG00000000460 0.893
#> 5 ENSG00000000971 0.0000206
#> 6 ENSG00000001036 0.0335
#> # ℹ 22,363 more rows
```

- Même famille que `count()` masqué par DESeq2 : lisez les messages `The following object is masked…` au chargement
- Écrire **explicitement** le package : `dplyr::select()` pour les colonnes, `AnnotationDbi::select()` pour les conversions

✋ À vous : prédire une conversion

```
ids <- c("ENSG00000152583.13", "ENSG00000004866", "ENSG00000096060")
conv <- AnnotationDbi::select(org.Hs.eg.db, keys = ids, keytype = "ENSEMBL", columns = "SYMBOL")
nrow(conv)
```

1. Combien de lignes ? Combien de `NA` ?
2. Même question avec `mapIds()`.
3. Vous aviez 3 gènes, vous écrivez `cbind(mes_resultats, conv)`. Que se passe-t-il ?

1. **4 lignes** : un `NA` (la version `.13` empêche la correspondance) et **deux** lignes pour `ENSG00000004866`
2. **3 valeurs**, une par clé, dans l'ordre : `NA`, `ST7` (le premier), `FKBP5`
3. Avec 3 lignes contre 4, `cbind()` refuse ; mais si les tailles tombaient juste, les lignes seraient **décalées** sans message. Joindre par l'identifiant (`left_join()`), et compter les lignes avant et après

TP1

5

L'ANALYSE DE SUR-REPRÉSENTATION

De la liste de gènes à la fonction

1018 gènes DE

- *SPARCL1* est associé à…
- *FKBP5* est associé à…
- *DUSP1* est associé à…
- …

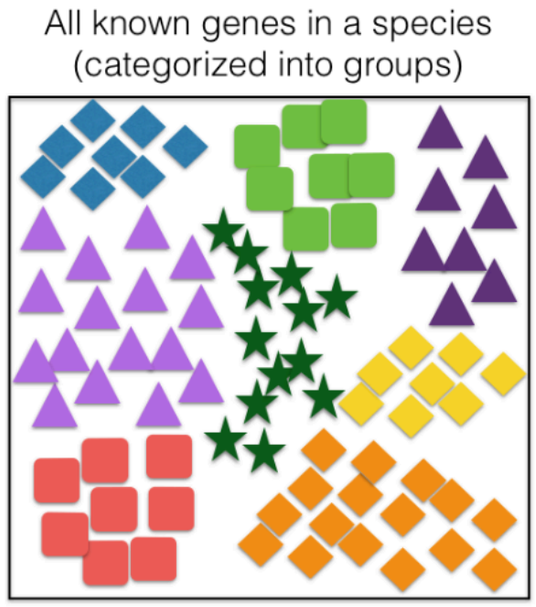
Parmi les gènes DE, y a-t-il des **thèmes communs** ?

Un thème « commun » dépend :

- de la **taille de la liste** de gènes DE
- du **nombre total** de gènes
- de la **taille du thème** (combien de gènes il contient)

11 gènes DE dans « *response to glucocorticoid* » : beaucoup ou peu ? Tout dépend de **combien on en attendait par hasard**.

L’urne





- Une urne de **N** gènes : l’**univers**
- **K** sont dans le terme GO (les étoiles)
- On tire **n** gènes : la liste DE
- On observe **k** étoiles dans le tirage

Si la liste DE était tirée **au hasard**, combien d’étoiles ?
En moyenne **n × K / N**.

La probabilité d’en voir **k ou plus** suit une loi **hypergéométrique** : c’est le **test exact de Fisher** unilatéral.

Le tableau de contingence

Chaque gène de l'univers : est-il **DE** ? est-il dans le **terme GO** ?

l'univers (extrait)

gene	DE	GO
FKBP5	oui	non
DUSP1	oui	oui
SPARCL1	oui	non
KLF9	oui	oui
ACTB	non	non
GAPDH	non	non
PER1	oui	non
MYH11	non	non

8 gènes de l'univers ; DE = induit (`padj` < 0.05, log2FC > 1) ; GO = annoté à *response to glucocorticoid* ou à l'un de ses descendants.

Le tableau de contingence

On compte les quatre cases, pour les 18 032 gènes testés et les 531 gènes **induits** :

response to glucocorticoid (GO:0051384)

	dans le terme	**hors du terme**	**total**
DE (induits)	11	520	531
non DE	81	17420	17501
total	92	17940	18032

- Observé : **k = 11** gènes induits dans le terme
- Attendu par hasard : n × K / N = 531 × 92 / 18 032 ≈ **2.7**
- 4.1 fois plus que le hasard. Est-ce significatif ?

Le test de Fisher

```
tab <- matrix(c(k, K - k, n - k, N - K - (n - k)), nrow = 2,
              dimnames = list(c("DE", "non DE"), c("dans", "hors")))
tab
#>         dans  hors
#> DE        11   520
#> non DE    81 17420
fisher.test(tab, alternative = "greater")$p.value
#> [1] 8.111353e-05
phyper(k - 1, K, N - K, n, lower.tail = FALSE)   # la même chose, avec la loi hypergéométrique
#> [1] 8.111353e-05
```

`alternative = "greater"` : on cherche une **sur**-représentation. Mais il y a des milliers de termes GO…

Des milliers de tests

- Un test de Fisher **par terme** : des milliers de termes GO testés
- Au seuil de 5 %, sans correction : des dizaines de termes « enrichis » **par hasard**
- Même remède qu'en expression différentielle : **Benjamini-Hochberg** (FDR), `p.adjust`
- Les termes ne sont pas indépendants (parents, enfants) : la correction reste approximative

> Deux étages de tests multiples : des milliers de **gènes** (DESeq2), puis des milliers de **termes** (ORA).

L'ORA dans R : enrichGO()

```
ora_induits <- enrichGO(gene     = induits$gene,       # la liste : les gènes induits
                        universe = testes$gene,       # l'univers : les gènes testés
                        OrgDb    = org.Hs.eg.db,
                        keyType  = "ENSEMBL",
                        ont      = "BP",              # Biological Process
                        pAdjustMethod = "BH",
                        readable = TRUE)              # afficher des symboles
```

```
as_tibble(ora_induits) |>
  dplyr::select(Description, GeneRatio, BgRatio, p.adjust, Count) |>
  head(6)
#> # A tibble: 6 × 5
#>   Description                                    GeneRatio BgRatio   p.adjust Count
#>   <chr>                                          <chr>     <chr>        <dbl> <int>
#> 1 muscle system process                          30/417    318/12376 0.000485    30
#> 2 extracellular matrix organization              26/417    254/12376 0.000485    26
#> 3 extracellular structure organization           26/417    254/12376 0.000485    26
#> 4 external encapsulating structure organization 26/417    256/12376 0.000485    26
#> 5 cell-substrate adhesion                        28/417    301/12376 0.000847    28
#> 6 regulation of system process                   32/417    376/12376 0.000862    32
```

Lire le résultat

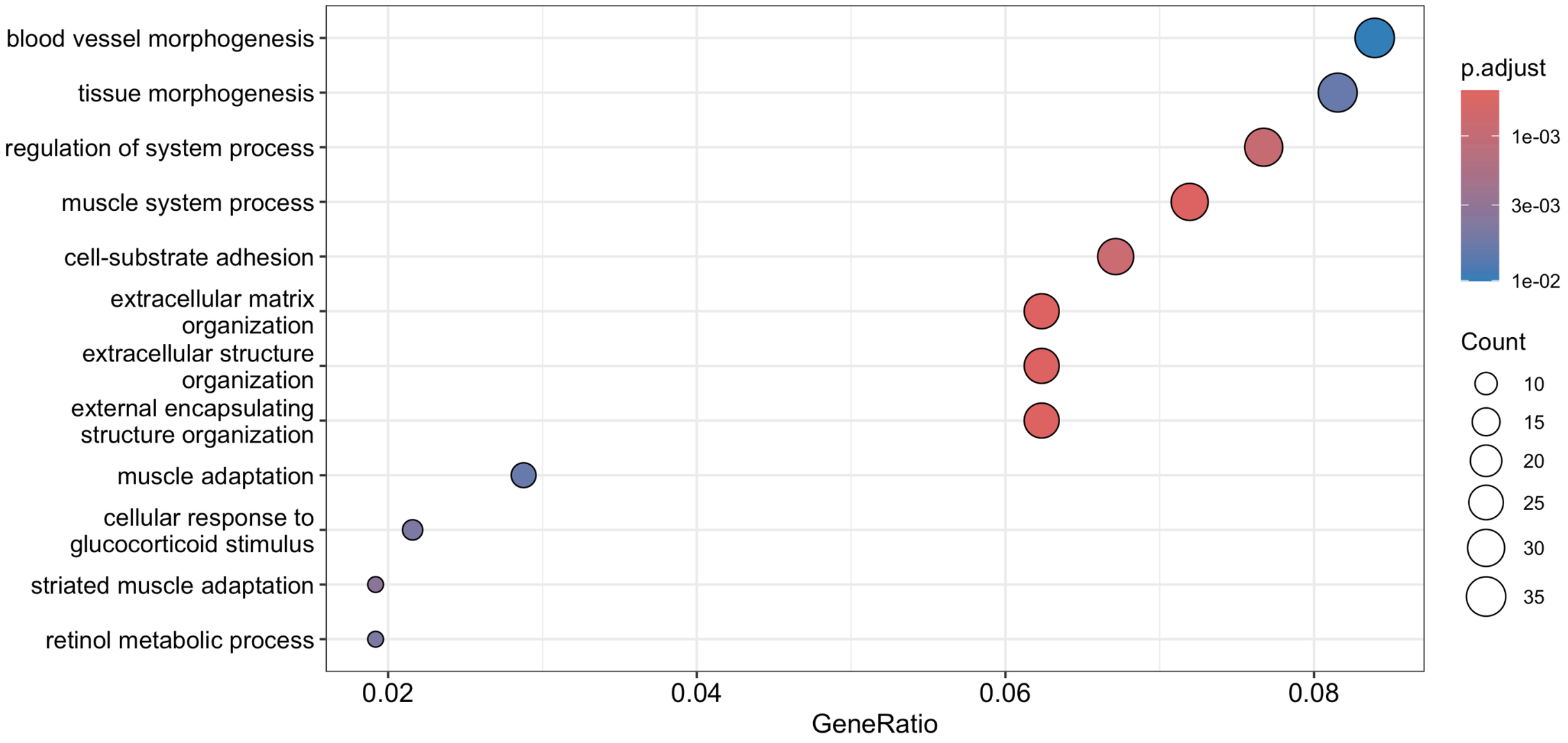
Colonne	Sens	*response to glucocorticoid*
`GeneRatio`	k / n : gènes de la liste dans le terme / gènes de la liste **annotés**	11/417
`BgRatio`	K / N : gènes de l'univers dans le terme / univers **annoté**	92/12376
`p.adjust`	p-value ajustée (BH)	0.021

`geneID` : les gènes de la liste dans le terme, ici CFLAR, IGFBP2, ERRFI1, FOXO3, KLF9, DUSP1, MSTN, FBXO32, STC1, FAM107A, AXIN2.

> Le dénominateur de `GeneRatio` n'est pas 531 : seuls les gènes qui ont **au moins une annotation GO BP** comptent. Les autres sont **retirés sans message**. N et n diffèrent donc de notre calcul à la main, et la p-value aussi.

Visualiser une ORA

```
dotplot(ora_induits, showCategory = 12) + theme(axis.text.y = element_text(size = 11))
```



Taille du point : nombre de gènes ; couleur : `p.adjust` ; x : `GeneRatio`.

L’univers change l’attendu

L’urne = les gènes **testés** : ceux qui **auraient pu** être DE.

response to glucocorticoid, gènes induits

urne	N	K	n	k	attendu	obs. / att.
gènes testés	12 376	92	417	11	3.1	3.5

N : gènes de l’urne ; K : dans le terme ; n : gènes de la liste ; k : de la liste dans le terme (gènes annotés seulement).

L'univers change l'attendu

Sans `universe` : on ajoute à l'urne des milliers de gènes **non exprimés**, qui ne pouvaient **pas** être tirés.

response to glucocorticoid, gènes induits

urne	N	K	n	k	attendu	obs. / att.
gènes testés	12 376	92	417	11	3.1	3.5
génome annoté	21 338	144	417	11	2.8	3.9

L'urne grossit (N), le terme un peu (K) : l'**attendu baisse**, le terme paraît **plus** enrichi. Même liste, même biologie, autre conclusion.

Piège : l'univers

```
ora_sans_univers <- enrichGO(gene = induits$gene, OrgDb = org.Hs.eg.db,
                             keyType = "ENSEMBL", ont = "BP")   # pas de 'universe'
```

- Sans `universe`, l'univers = **tous** les gènes humains annotés : 21 338 au lieu de 12 376
- Or un gène **non exprimé** dans ces cellules ne pouvait **pas** être DE : on le compte dans l'urne alors qu'il n'avait aucune chance d'être tiré
- Résultat : 161 termes significatifs au lieu de 94. Un terme dont les gènes sont simplement **exprimés** dans ces cellules paraît enrichi, sans rapport avec le **traitement**

> L'univers = les gènes qui **auraient pu** être dans la liste : les gènes **testés** (`padj` non `NA`). Les outils web l'oublient presque tous par défaut (Timmons *et al.*, 2015 ; Wijesooriya *et al.*, 2022).

Piège : mélanger induits et réprimés

Liste	Gènes	Termes BP significatifs
induits	531	94
réprimés	487	372
les deux ensemble	1018	531

- Ensemble, un terme « enrichi » ne dit plus **dans quel sens** la fonction change
- 110 termes significatifs dans un sens **disparaissent** dans la liste commune, dilués, dont *response to glucocorticoid* ; 194 termes n'**apparaissent** que dans la liste commune
- Faire **une ORA par sens**, et le dire

> « Enrichi » ne veut jamais dire « activé ». Un terme enrichi parmi les gènes induits peut contenir des **inhibiteurs** de la voie.

Pourquoi le terme disparaît

Les gènes **induits** seuls : 11 dans le terme, 3,5 fois l'attendu.

response to glucocorticoid

liste	n	k	attendu	obs. / att.	p.adjust
induits seuls	417	11	3.1	3.5	0.021

Pourquoi le terme disparaît

On ajoute les **réprimés** : la liste double, mais 1 gène de plus dans le terme. L'attendu double, le signal est **dilué**.

response to glucocorticoid

liste	n	k	attendu	obs. / att.	p.adjust
induits seuls	417	11	3.1	3.5	0.021
induits + réprimés	818	12	6.1	2	0.1

Les outils en ligne

Outil	Adresse	Particularité
g:Profiler	https://biit.cs.ut.ee/gprofiler	GO, voies, facteurs de transcription ; version de la base indiquée ; aussi en R (`gprofiler2`)
Enrichr	https://maayanlab.cloud/Enrichr	des centaines de collections (maladies, tissus, médicaments)
ToppGene	https://toppgene.cchmc.org	priorisation de gènes
DAVID	https://davidbioinformatics.nih.gov	historique, très cité

Rapide et pratique, mais : préciser l'**univers**, noter la **version** des annotations et la **date**, et vérifier la **correction** des tests multiples. Une analyse faite sur un site web en 2026 ne se refera pas à l'identique en 2027.

Les limites de l'ORA

- Il faut un **seuil** pour faire la liste (`padj` < 0.05, |log2FC| > 1) : arbitraire, et les résultats en dépendent
- Un gène juste sous le seuil compte **autant** qu'un gène non exprimé ; un gène à log2FC = 8 compte **autant** qu'un gène à log2FC = 1,1
- Aucune liste si l'effet est faible et diffus (des dizaines de gènes qui bougent un peu)
- Suppose les gènes **indépendants** ; or les gènes d'une voie sont souvent co-régulés
- **Biais de longueur** en RNA-seq : les gènes longs ont plus de lectures, donc plus de chances d'être DE (`goseq` le corrige)

> La GSEA répond aux trois premières limites : **pas de seuil**, tous les gènes, classés.

✋ À vous : lire une ORA

Un étudiant envoie sa liste de 400 gènes réprimés sur un site web, avec les réglages par défaut, et obtient :

term	GeneRatio	BgRatio	p-value
extracellular matrix organization	24/312	330/19800	2.1e-06

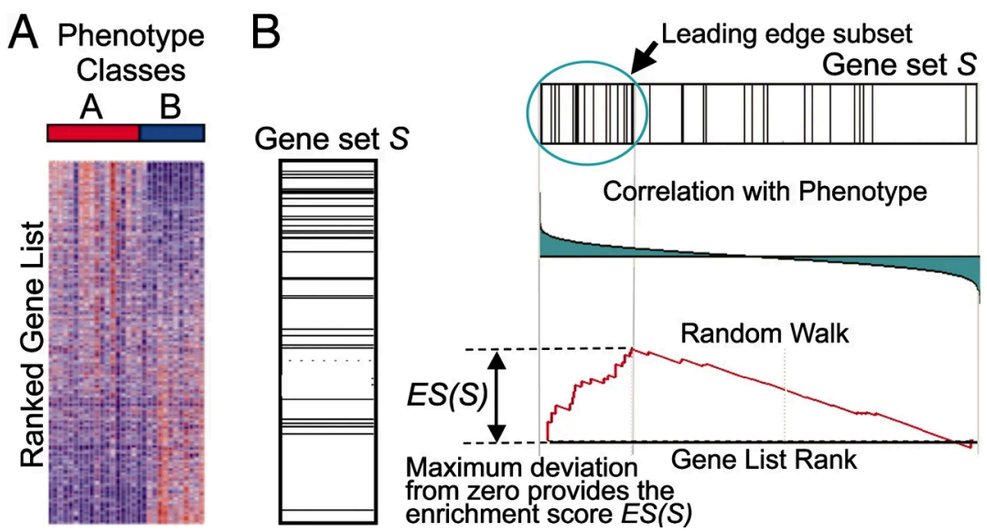
1. Combien de gènes de la liste sont dans le terme ? Combien en attendait-on par hasard ?
2. Pourquoi 312 et non 400 ?
3. Quel est l'univers utilisé ? Est-ce un problème ?
4. Il conclut : « le traitement active l'organisation de la matrice extracellulaire ». Deux objections.

1. **24** ; attendu 312 × 330 / 19 800 ≈ **5,2**
2. 88 gènes n'ont **aucune annotation** dans cette ontologie : retirés en silence
3. Tous les gènes annotés du génome (**19 800**), pas les gènes testés : enrichissement probablement **surestimé**
4. Une **p-value brute** (des milliers de termes testés : il faut `p.adjust`) ; et des gènes **réprimés** ne disent pas « active ». Enrichi ≠ activé

6

LA GSEA

L'idée



Gene Set Enrichment Analysis

- **Tous** les gènes, pas seulement les DE : **aucun seuil**
- Les gènes sont **classés** selon une statistique (ici la `stat` de Wald de DESeq2)
- Question : les gènes d'un ensemble sont-ils **regroupés en haut** (ou en bas) du classement, plutôt que répartis au hasard ?

Subramanian *et al.* (2005), *PNAS*.

> Un ensemble dont les gènes montent tous **un peu**, sans qu'aucun ne passe le seuil, est invisible pour l'ORA. La GSEA le voit : ses gènes sont regroupés en haut du classement.

Classer les gènes

```
classement <- testes |> arrange(desc(stat))
liste_gsea <- setNames(classement$stat, classement$gene)   # un vecteur nommé, trié
head(liste_gsea, 4)
#> ENSG00000152583 ENSG00000165995 ENSG00000120129 ENSG00000101347
#>          24.831          24.735          24.187          24.149
tail(liste_gsea, 4)
#> ENSG00000107562 ENSG00000148848 ENSG00000178695 ENSG00000162692
#>         -17.985         -18.010         -18.750         -19.428
```

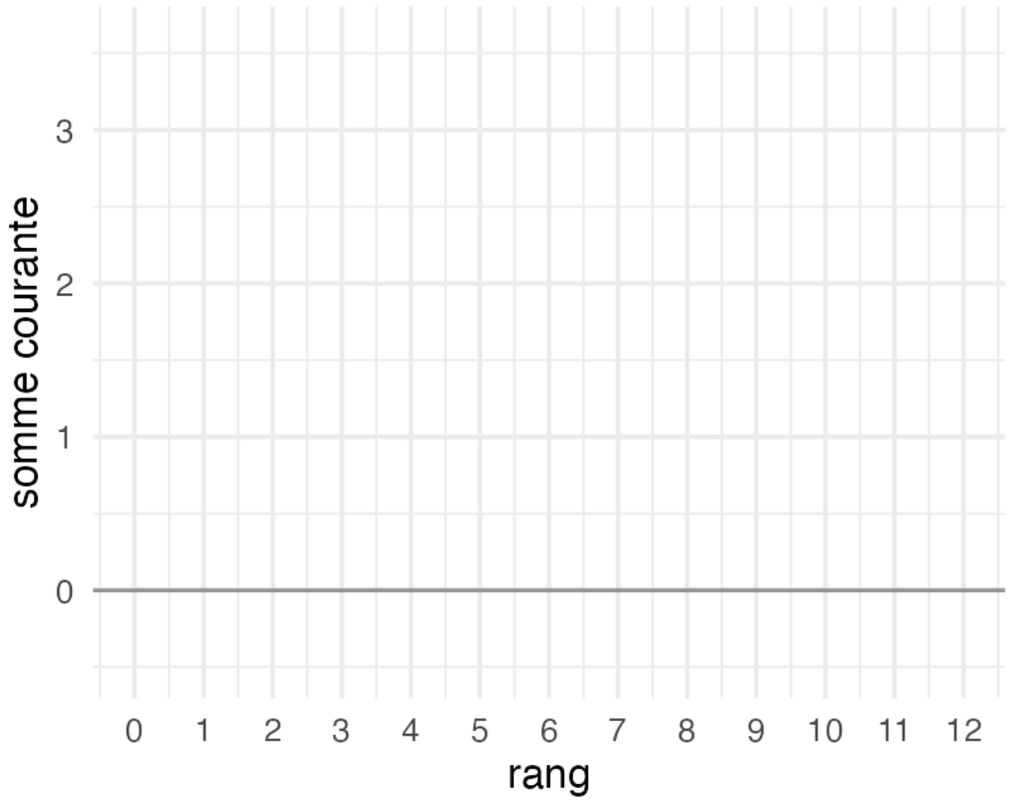
- En haut : les plus **induits**, avec certitude ; en bas : les plus **réprimés**
- `stat` = log2FC / lfcSE : combine l'**effet** et la **précision**
- Le classement a un **sens** : tout dépend du niveau de référence (`dex trt vs untrt`)

La somme courante

On parcourt la liste classée **de haut en bas**. La somme part de **0**.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui		
2	g2	oui		
3	g3	non		
4	g4	oui		
5	g5	non		
6	g6	oui		
7	g7	non		
8	g8	non		
9	g9	oui		
10	g10	non		
11	g11	non		
12	g12	non		

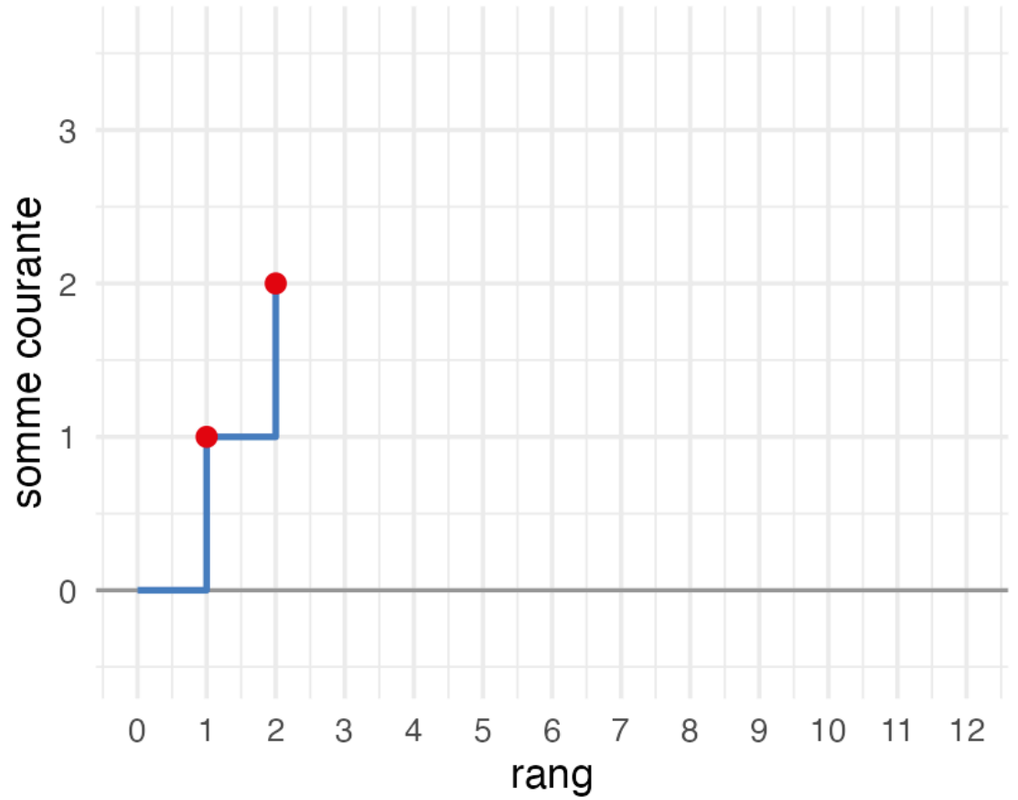


La somme courante

g1 et g2 sont dans l'ensemble S : **+1**, **+1**.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui	+1	1.0
2	g2	oui	+1	2.0
3	g3	non		
4	g4	oui		
5	g5	non		
6	g6	oui		
7	g7	non		
8	g8	non		
9	g9	oui		
10	g10	non		
11	g11	non		
12	g12	non		

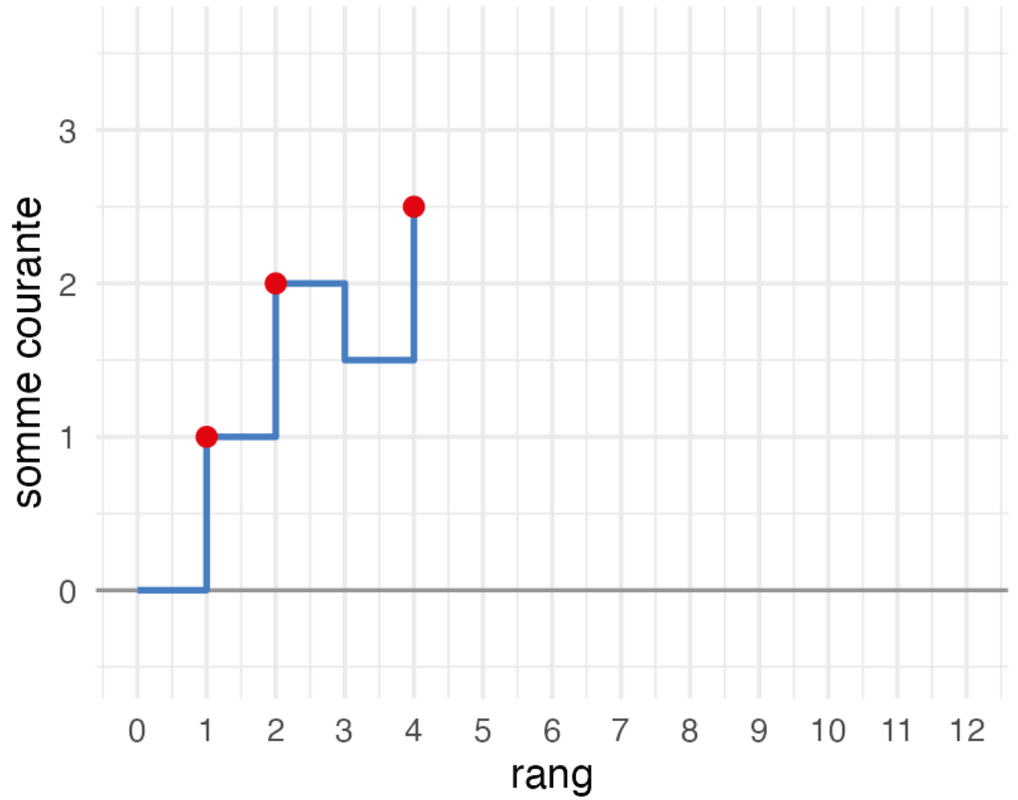


La somme courante

g3 n'est pas dans S : **−0,5**. g4 l'est : **+1**.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui	+1	1.0
2	g2	oui	+1	2.0
3	g3	non	–0,5	1.5
4	g4	oui	+1	2.5
5	g5	non		
6	g6	oui		
7	g7	non		
8	g8	non		
9	g9	oui		
10	g10	non		
11	g11	non		
12	g12	non		

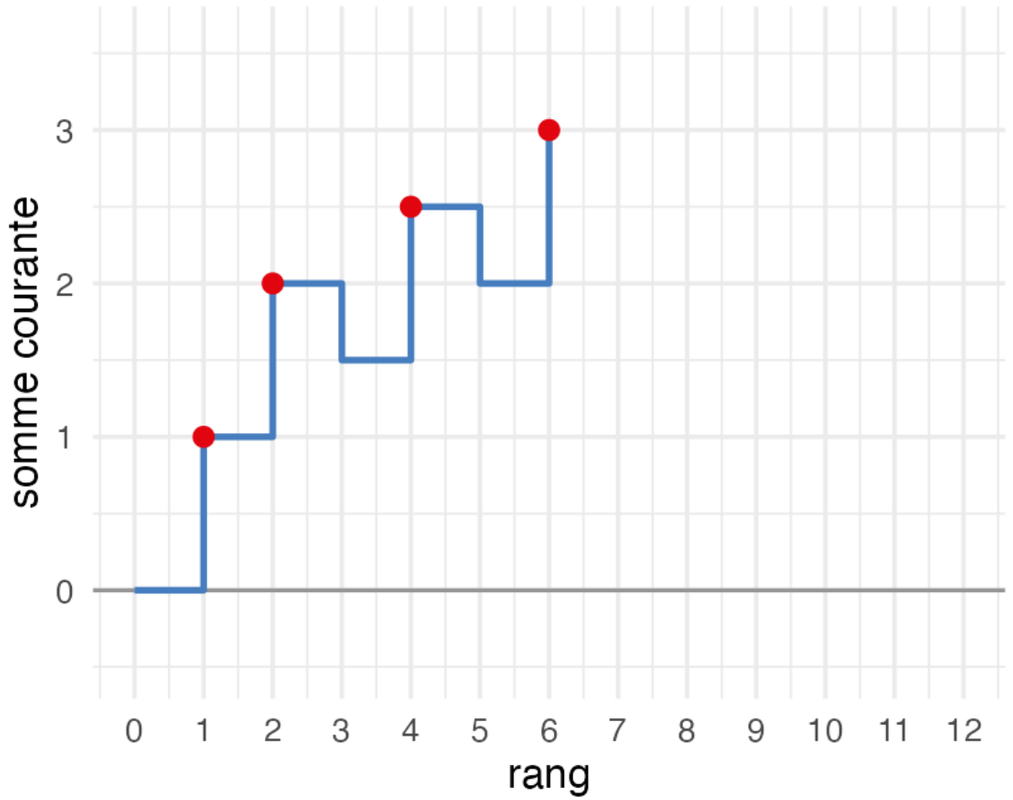


La somme courante

Avec g6, la somme atteint **3** : son maximum.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui	+1	1.0
2	g2	oui	+1	2.0
3	g3	non	–0,5	1.5
4	g4	oui	+1	2.5
5	g5	non	–0,5	2.0
6	g6	oui	+1	3.0
7	g7	non		
8	g8	non		
9	g9	oui		
10	g10	non		
11	g11	non		
12	g12	non		

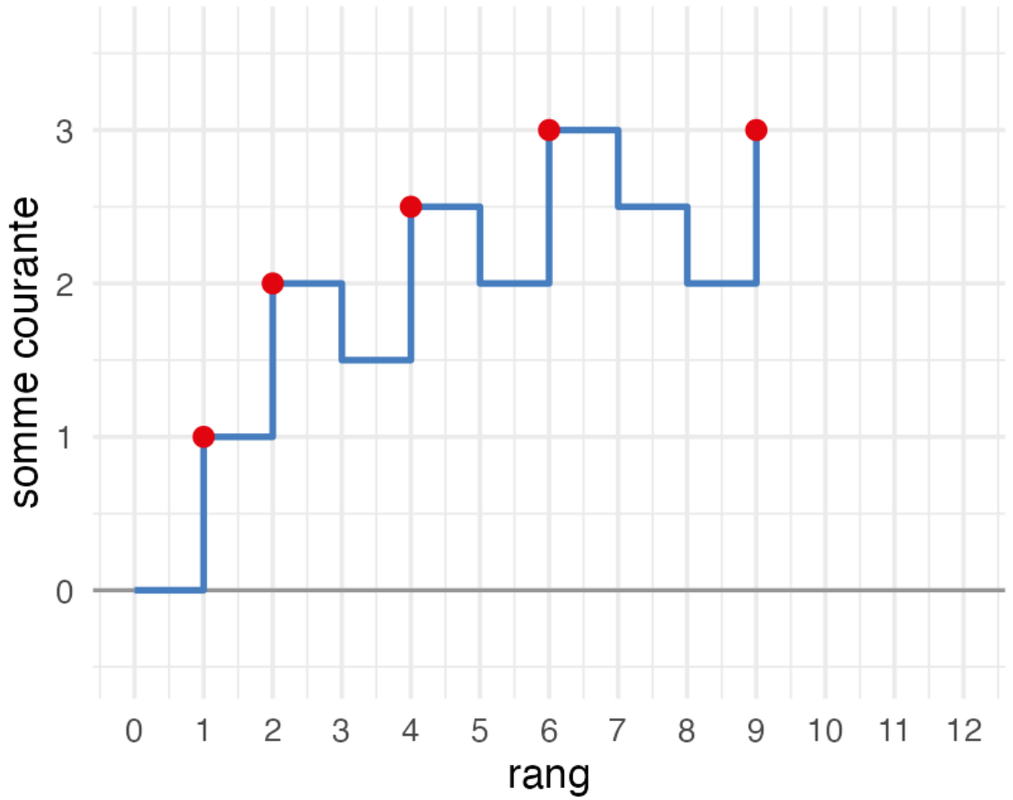


La somme courante

Les gènes hors de S la font redescendre ; g9 la remonte à 3.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui	+1	1.0
2	g2	oui	+1	2.0
3	g3	non	–0,5	1.5
4	g4	oui	+1	2.5
5	g5	non	–0,5	2.0
6	g6	oui	+1	3.0
7	g7	non	–0,5	2.5
8	g8	non	–0,5	2.0
9	g9	oui	+1	3.0
10	g10	non		
11	g11	non		
12	g12	non		

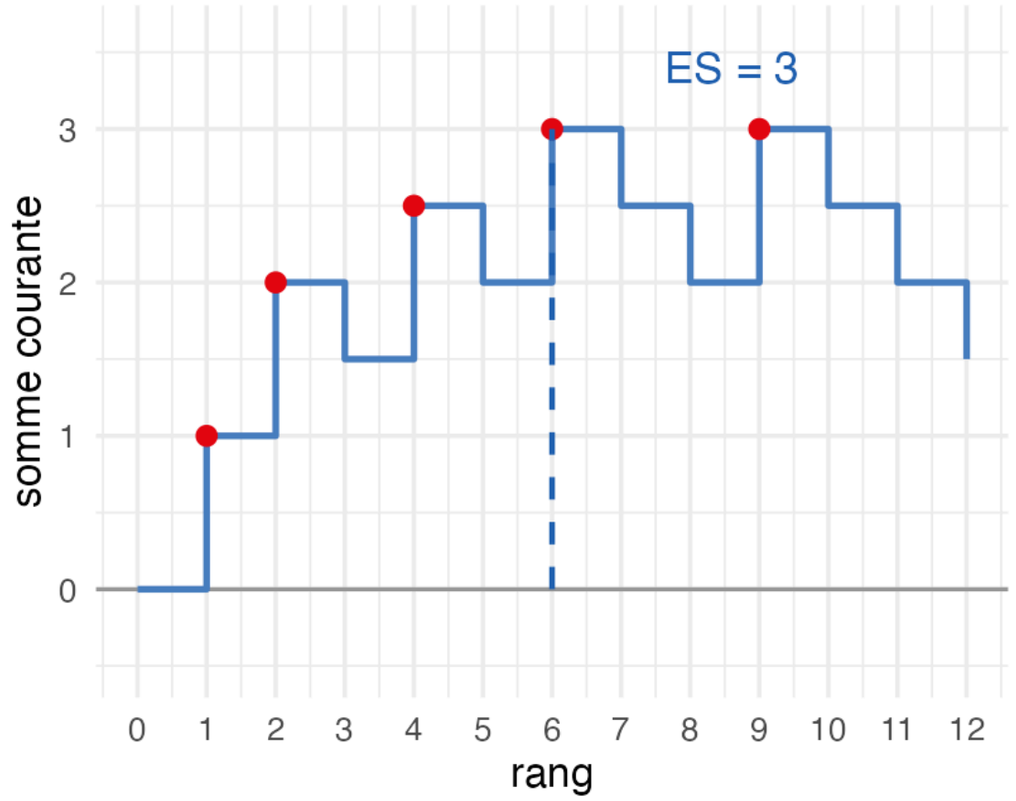


La somme courante

Fin de la liste. **ES = 3**, l'écart maximal à zéro. Positif : S est en **haut** du classement.

liste classée (jouet)

rang	gene	dans S	pas	somme
1	g1	oui	+1	1.0
2	g2	oui	+1	2.0
3	g3	non	–0,5	1.5
4	g4	oui	+1	2.5
5	g5	non	–0,5	2.0
6	g6	oui	+1	3.0
7	g7	non	–0,5	2.5
8	g8	non	–0,5	2.0
9	g9	oui	+1	3.0
10	g10	non	–0,5	2.5
11	g11	non	–0,5	2.0
12	g12	non	–0,5	1.5



Le vrai calcul

- Les pas sont **pondérés** : un gène de S monte d'autant plus que sa `stat` est forte ; les pas des autres gènes sont calculés pour que la somme revienne à **0** à la fin
- La statistique ressemble à celle de **Kolmogorov-Smirnov** : l'écart maximal entre deux distributions cumulées
- **Significativité** : on **permute** (les gènes ou les échantillons) des milliers de fois et on recalcule l'ES. La p-value = la proportion d'ES permutés au moins aussi extrêmes
- **NES** (*normalized ES*) : l'ES divisé par la moyenne des ES permutés, pour comparer des ensembles de **tailles** différentes
- Puis correction pour les tests multiples (BH) sur tous les ensembles

> Les **gènes moteurs** (*leading edge*) : les gènes de S situés **avant** le pic. Ce sont eux qui portent l'enrichissement.

La GSEA dans R : `gseGO()`

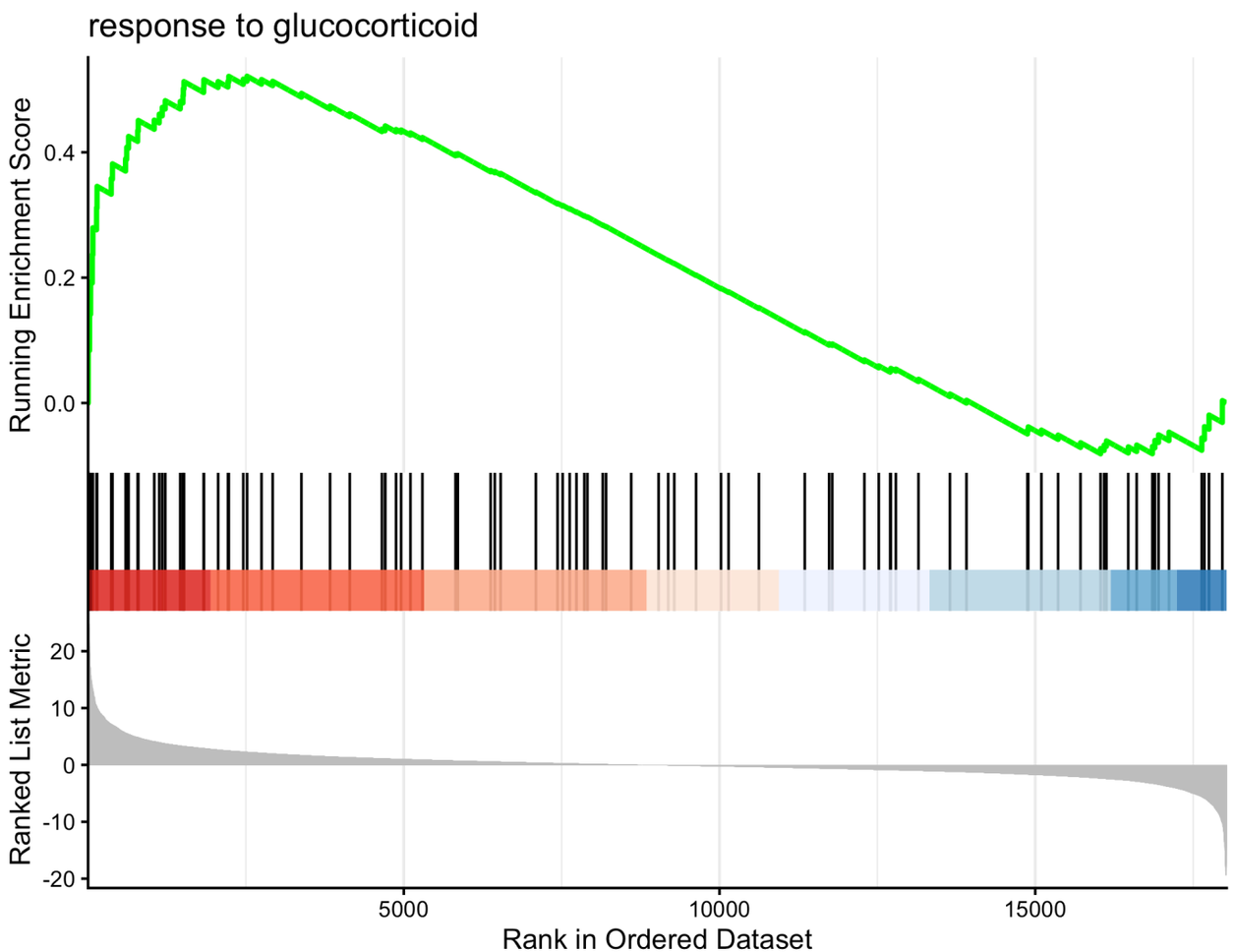
```
gsea_go <- gseGO(geneList = liste_gsea,      # le vecteur nommé et trié
                 OrgDb    = org.Hs.eg.db,
                 keyType  = "ENSEMBL",
                 ont      = "BP",
                 minGSSize = 15, maxGSSize = 500,   # tailles d'ensembles retenues
                 seed     = TRUE)                    # permutations reproductibles
```

```
as_tibble(gsea_go) |>
  dplyr::select(Description, setSize, NES, p.adjust) |>
  arrange(desc(NES)) |> head(4)
#> # A tibble: 4 × 4
#>   Description                          setSize   NES p.adjust
#>   <chr>                                  <int> <dbl>    <dbl>
#> 1 D-glucose import                          63  2.16 0.000107
#> 2 regulation of D-glucose import            46  2.15 0.000566
#> 3 epithelial cell morphogenesis             25  2.10 0.00300
#> 4 D-glucose transmembrane transport         83  2.06 0.000566
```

`There are ties in the preranked stats` : des gènes à égalité dans le classement, dont l'ordre sera arbitraire. Un avertissement, pas une erreur.

Le graphique de GSEA

```
gseaplot2(gsea_go, geneSetID = "GO:0051384",
          title = "response to glucocorticoid")
```



- **En haut** : la somme courante ; son pic = l'ES
- **Au milieu** : chaque trait = un gène de l'ensemble, à son rang
- **En bas** : la statistique de classement
- Ici : NES = 1.83, `p.adjust` = 0.0092 ; l'ensemble est en **haut** : **induit** par la dexaméthasone

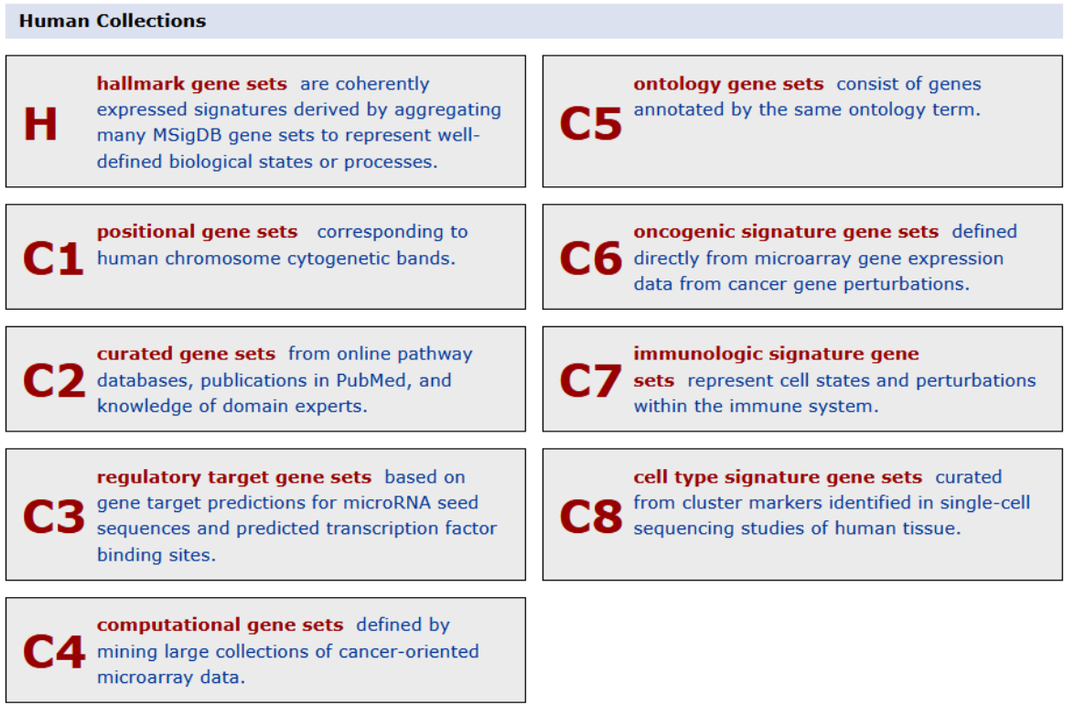
Lire le signe du NES

```
as_tibble(gsea_go) |> dplyr::count(sens = if_else(NES > 0, "NES > 0", "NES < 0"))
#> # A tibble: 2 × 2
#>   sens         n
#>   <chr>    <int>
#> 1 NES < 0     99
#> 2 NES > 0    211
```

- **NES > 0** : l'ensemble est en haut du classement, ses gènes tendent à être **plus** exprimés chez les traités
- **NES < 0** : en bas, **moins** exprimés chez les traités

> Le signe vient du classement, donc du **niveau de référence** de DESeq2. Avec `untrt vs trt`, **tous** les NES changent de signe : même erreur silencieuse qu'en statistiques. Lire l'en-tête de `res` **avant** l'enrichissement.

Où trouver des ensembles de gènes ?

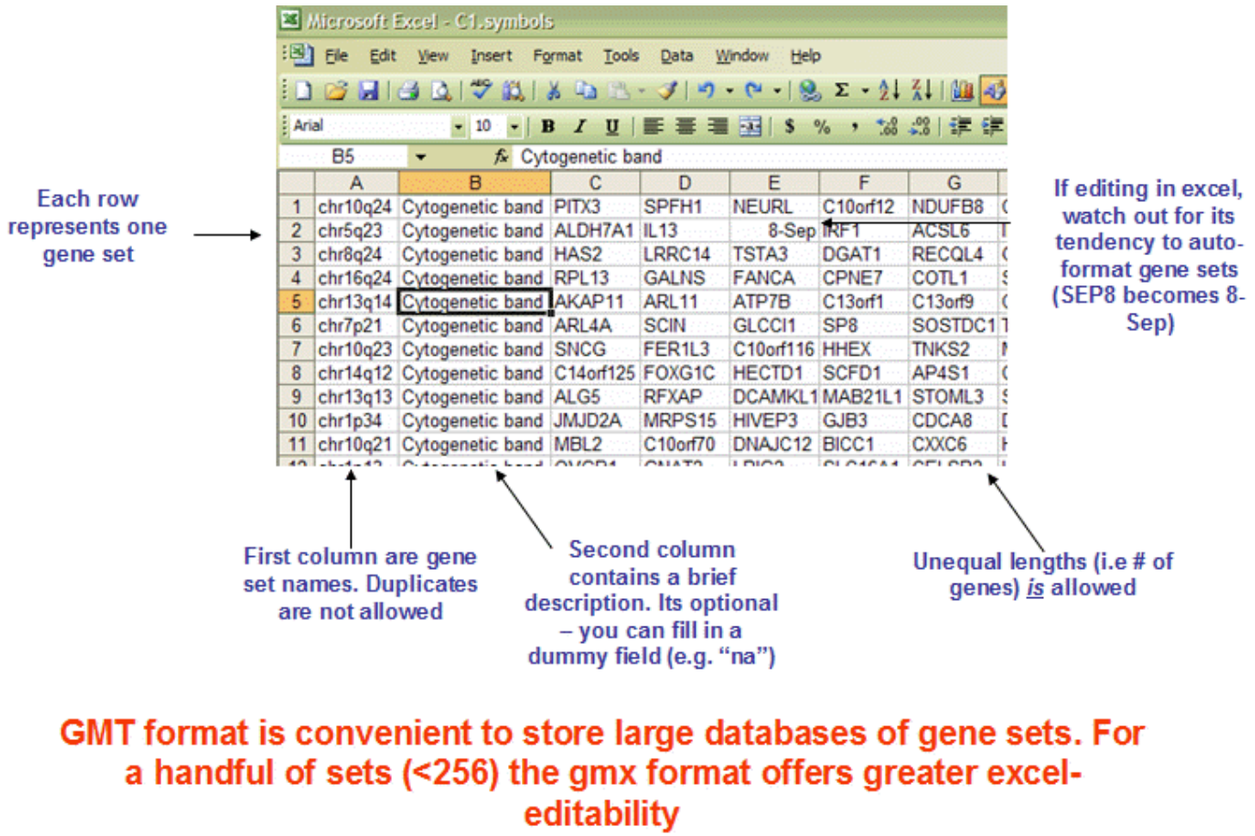


MSigDB (*Molecular Signatures Database*, Broad Institute)

- Des collections : **H** (*hallmarks*), C2 (voies : KEGG, Reactome), C5 (GO), C7 (immunologie)…
- **Hallmarks** : 50 ensembles de référence, peu redondants, chacun résumant un processus bien défini (Liberzon *et al.*, 2015)
- Dans R : `msigdbr`, ou un fichier GMT téléchargé

Et aussi : GO, KEGG, Reactome, WikiPathways, Disease Ontology… La GSEA accepte **n'importe quel** ensemble de gènes.

Le format GMT



- *Gene Matrix Transposed* : une ligne par ensemble ; le nom, une description, puis les gènes
- Lecture : `read.gmt()` (`clusterProfiler`) → un tableau à deux colonnes `term`, `gene`
- Analyse avec ses propres ensembles : `GSEA(liste, TERM2GENE = tableau)` ou `enricher()` pour une ORA

Ouvrir un GMT dans Excel : les symboles *SEPT2*, *MARCH1*… redeviennent des dates.

Les Hallmarks : un bon point de départ

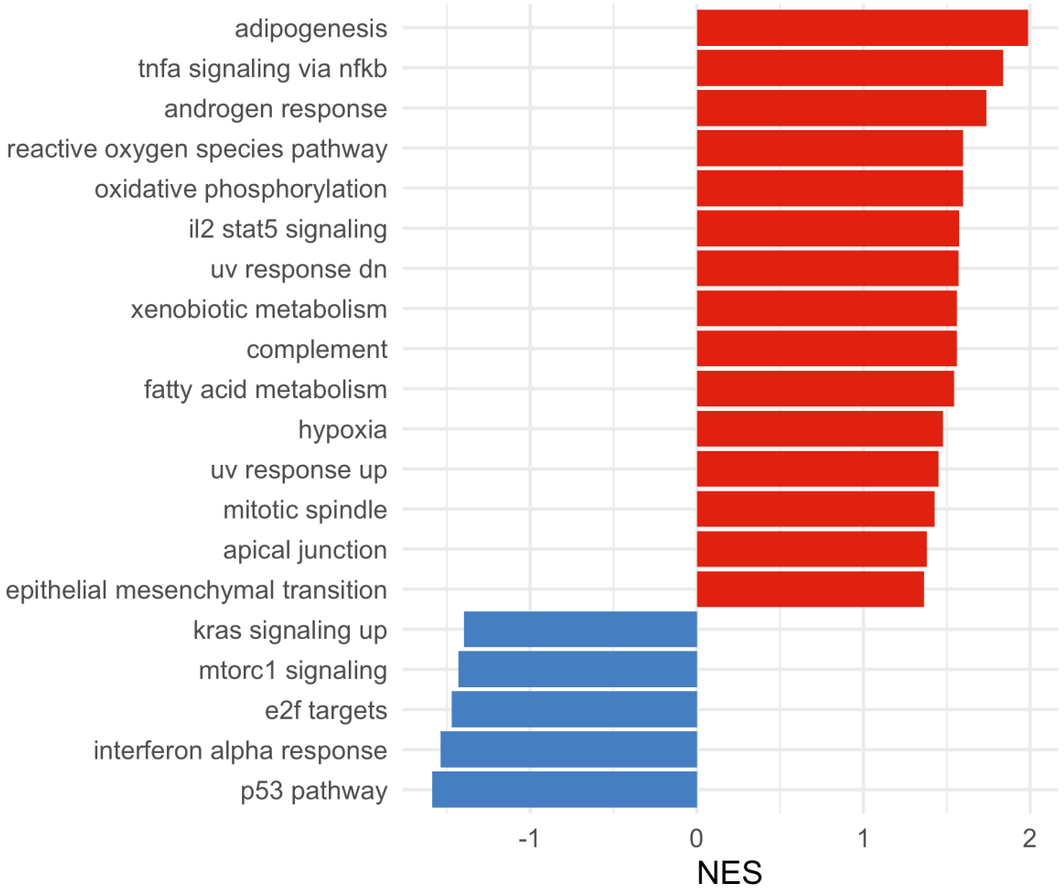
```
hallmarks <- read.gmt("/home/public/enrichissement/h.all.Hs.ensembl.gmt")
```

```
head(hallmarks, 3)
#>                    term            gene
#> 1 HALLMARK_ADIPOGENESIS ENSG00000165029
#> 2 HALLMARK_ADIPOGENESIS ENSG00000197150
#> 3 HALLMARK_ADIPOGENESIS ENSG00000167315
gsea_h <- GSEA(geneList = liste_gsea, TERM2GENE = hallmarks, seed = TRUE)
```

```
as_tibble(gsea_h) |> dplyr::select(ID, setSize, NES, p.adjust) |> arrange(desc(NES)) |> head(5)
#> # A tibble: 5 × 4
#>   ID                                  setSize   NES   p.adjust
#>   <chr>                                 <int> <dbl>      <dbl>
#> 1 HALLMARK_ADIPOGENESIS                   191  1.99 0.00000309
#> 2 HALLMARK_TNFA_SIGNALING_VIA_NFKB        175  1.84 0.0000794
#> 3 HALLMARK_ANDROGEN_RESPONSE               92  1.74 0.00632
#> 4 HALLMARK_REACTIVE_OXYGEN_SPECIES_PATHWAY  48  1.60 0.0240
#> 5 HALLMARK_OXIDATIVE_PHOSPHORYLATION       198  1.60 0.00632
```

MSigDB 2026.1.Hs, 50 ensembles. Le même vecteur classé que pour `gseGO()` ; seuls les ensembles changent.

Lire les Hallmarks

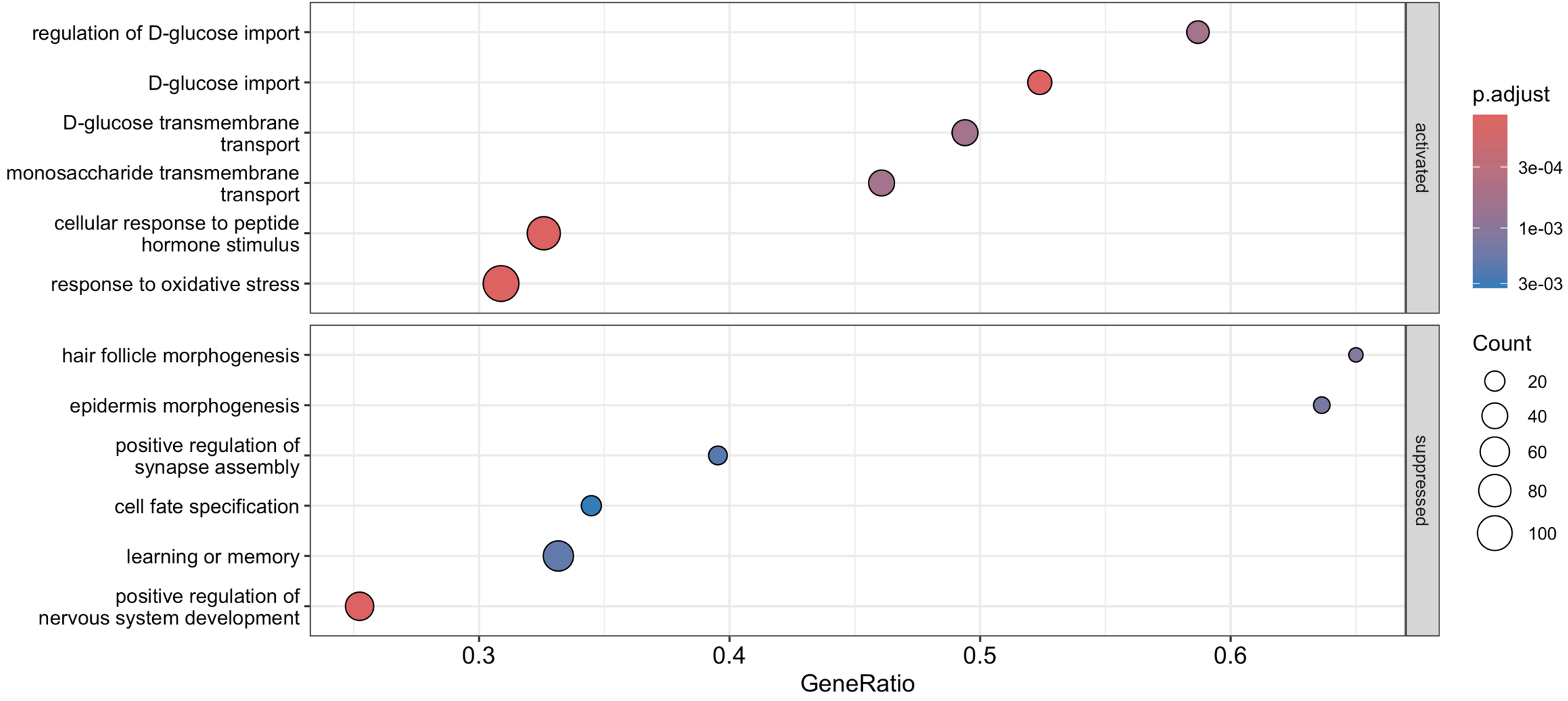


- 20 Hallmarks sur 50 significatifs, **peu redondants** : 50 ensembles au lieu de milliers de termes GO
- **Adipogenèse** en tête : la dexaméthasone fait partie des cocktails de différenciation des adipocytes
- **TNFα via NF-κB, NES > 0** : la dexaméthasone, un anti-inflammatoire, « active » l'inflammation ?

Non : parmi ses gènes moteurs, *DUSP1*, *NFKBIA*, *TNFAIP3*, *ZFP36*, *TNIP1*, *TNIP2*, des **freins** de NF-κB induits par les glucocorticoïdes. **Enrichi ≠ activé** : lire les gènes.

Visualiser plusieurs ensembles

```
dotplot(gsea_go, showCategory = 6, split = ".sign") + facet_grid(.sign ~ ., scales = "free_y") +
  theme(axis.text.y = element_text(size = 10))
```



`activated` / `suppressed` : le vocabulaire de `clusterProfiler` pour NES > 0 / NES < 0. Ce sont des tendances d'expression, pas des mesures d'activité.

ORA ou GSEA ?

	ORA	GSEA
Entrée	une **liste** de gènes (et l'univers)	**tous** les gènes, **classés**
Seuil	oui, arbitraire	non
Test	hypergéométrique (Fisher)	permutations, ES pondéré
Sens de variation	une analyse par sens	dans le signe du NES
Détecte	quelques gènes à effet fort	beaucoup de gènes à effet modéré
Quand ?	une liste sans statistique (protéomique, gènes candidats, CRISPR)	un résultat d'expression différentielle complet

Les deux reposent sur les **mêmes annotations** : leurs résultats ne valent que ce que valent la base, sa version et ses codes d'évidence.

✋ À vous : lire une GSEA

```
#> # A tibble: 4 × 5
#>   ID         Description                        setSize   NES p.adjust
#>   <chr>      <chr>                                <int> <dbl>    <dbl>
#> 1 GO:0046323 D-glucose import                        63  2.16 0.000107
#> 2 GO:0046324 regulation of D-glucose import          46  2.15 0.000566
#> 3 GO:0031069 hair follicle morphogenesis             20 -2.25 0.00110
#> 4 GO:0048730 epidermis morphogenesis                 22 -2.21 0.00148
```

1. Quels processus sont plutôt **plus** exprimés chez les cellules traitées ? Lesquels **moins** ?
2. Une collègue refait l'analyse ; sa première ligne de `res` dit `dex untrt vs trt`. Que devient la colonne `NES` ?
3. Elle trie sa liste par ordre **croissant** de `stat`. Que se passe-t-il ?
4. Un ensemble de 8 gènes n'apparaît nulle part. Pourquoi ?

1. NES > 0 : D-glucose import, regulation of D-glucose import ; NES < 0 : hair follicle morphogenesis, epidermis morphogenesis
2. **Tous les signes s'inversent**, p-values quasi identiques : seule la lecture de l'en-tête le révèle
3. Une **erreur**, visible cette fois : `geneList should be a decreasing sorted vector`
4. `minGSSize = 15` : les ensembles de moins de 15 gènes ne sont **pas testés**

TP2

Conclusion

- Des **milliers** de bases : primaires (données brutes déposées) ou secondaires (interprétées, parfois prédites) ; vérifier la **curation** et la **version**
- Les **ontologies** (GO, MeSH) donnent un vocabulaire contrôlé ; GO est un graphe, ses annotations portent un **code d'évidence**
- Les bases sont reliées par leurs **identifiants** ; convertir, c'est utiliser une base de plus : **compter** les lignes, les NA, les doublons
- **ORA** : une liste, un univers, un test de Fisher par terme, une correction BH ; une analyse **par sens**
- **GSEA** : tous les gènes classés, un score par ensemble, des permutations ; le **signe** du NES dépend de la référence
- Noter les versions : `sessionInfo()`, `AnnotationDbi::metadata(org.Hs.eg.db)`

Ressources

- *clusterProfiler* et *enrichplot* : le livre *Biomedical Knowledge Mining using GOSemSim and clusterProfiler* (G. Yu) : https://yulab-smu.top/biomedical-knowledge-mining-book/
- Atelier HBC, *Functional analysis* : https://hbctraining.github.io/DGE_workshop/lessons/09_functional_analysis.html
- Gene Ontology : https://geneontology.org ; MSigDB : https://www.gsea-msigdb.org
- Reimand *et al.* (2019), *Pathway enrichment analysis and visualization of omics data*, *Nature Protocols* : le guide pas à pas
- Wijesooriya *et al.* (2022), *Urgent need for consistent standards in functional enrichment analysis*, *PLoS Computational Biology* : les erreurs fréquentes dans les articles publiés

Données : `airway` (Himes *et al.*, 2014 ; GEO GSE52778), résultats DESeq2 du module de statistiques.